

SF Crime Classification: Spatial-Temporal Machine Learning

Alec Olvera

Multiclass crime prediction using logistic regression,
spatial clustering, cyclical temporal encoding,
and probabilistic modeling on the San Francisco Crime Dataset.

This project studies a dataset regarding crime incidents in San Francisco between 2003 and 2015. Our goal is to predict the crime category from the features within the dataset: temporal features, geographic location, police district, and some engineered spatial-temporal predictors which will be explained further on. This is difficult because of the nature of this dataset. There are several overlapping crime patterns, high class imbalance, strong spatial variation, and nonlinear relationships. So, crimes occurring in similar locations may belong to very different categories depending on the hour, district, and neighborhood context. This dataset contains features like timestamp (Dates), response variables, police district, address, longitude/latitude, and the day of the week. This is also a large dataset, with around 878k observations total and 39 crime classes, which makes computation expensive and validation difficult. For my training and test split I used stratified splitting in an attempt to preserve category proportions across splits. The primary metric for this model is to minimize multiclass log loss. This is because accuracy in this context is insufficient as a metric. This is because in this context model confidence matters, so we need incorrect high-confidence predictions to be heavily penalized. Probability calibration matters in multiclass problems, so log loss becomes the primary metric. I will be starting with some basic data cleaning: data parsing, address parsing, geographic filtering; then feature engineering: cyclical encoding, interaction terms, geographic clustering; then model development: logistic regression and categorical Naïve Bayes; and then validation. This project

is ultimately meant to demonstrate that incorporating spatial clustering and cyclical temporal structure substantially improves predictive performance over baseline linear models.

During initial data cleaning, I removed Descript and Resolution since these variables contain post-event information unavailable at the prediction time and would artificially inflate the model performance. Some observations within the longitude values were removed since they contained unrealistic coordinates, distorting geographic modeling and clustering methods. Also, I standardized categorical/string preprocessing. After cleaning the raw observations, temporal and geographical variables were expanded into engineered predictors better suited for the upcoming models.

One thing I noticed was crime is highly time dependent. Different crimes peak at different hours, weekdays, seasons, months. From these observations, I derived the following temporal features: year, month, day, hour, weekday, week of year. Raw datetime strings cannot be directly interpreted by linear classification models, require decomposition into structured numerical predictors. Below is a plot visualizing this phenomenon:

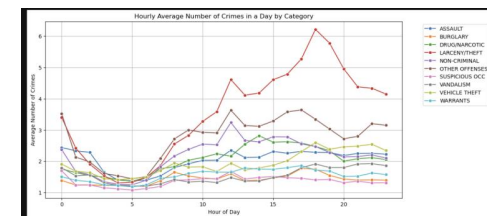


Figure #1: Hours Average Number of Crimes in a Day by Category

As can be seen, theft-related crimes peak during evening hours, and drug/narcotic crimes rise during daytime. So, these temporal periodicities motivated the use of cyclical feature transformations described later.

Next was tackling the issue of address and spatial features. Raw address strings are unstructured here, and we need to extrapolate meaningful categorical variables and geographic interaction information. So, with intersections I created a binary intersection feature. This is because intersections often correspond to commercial/high traffic regions and may correlate with certain crime types. For streets, I parsed address into street components and created encoded street level predictors. Regarding police districts, I used one-hot encoding, which allows the districts to provide coarse geographic segmentation. Crime distributions differ substantially across SF neighborhoods and districts, making spatial feature engineering critical for predictive performance.

After addressing the previous features, I wanted to try the idea of cyclical coding. Linear models treat hour 23 and hour 0 as numerically far apart, but they are adjacent in time. So, I used two formulas in order to simulate this effect where $x_{\sin} = \sin(2\pi x/T)$ and then $x_{\cos} = \cos(2\pi x/T)$ where x is the temporal variable and T is the cycle length. So, for a hour, $T = 24$, weekday $T = 7$, etc. Cyclical encoding produced one of the largest non-spatial improvements in validation.

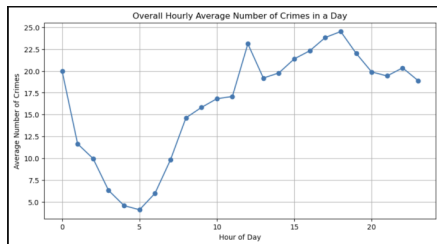


Figure #2: Overall Hourly Average Number of Crimes in a Day

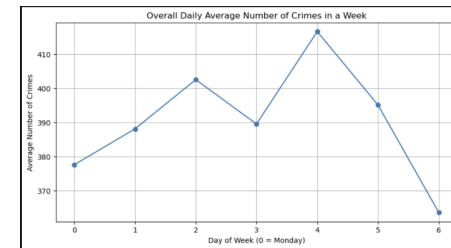


Figure #3: Overall Daily Average Number of Crimes in a Week

After this, I ran into the issue in of raw latitude/longitude coordinated being difficult for linear classifiers to interpret directly. This motivated me to try two strategies: MiniBatchKMeans clustering and to partition geographic regions into localized spatial zones. Geographic clustering should allow the model to learn localized crime tendencies while reducing sensitivity to raw coordinate noise. I also experimented with 5-40 clusters, and noticed performance improved as spatial granularity increased.

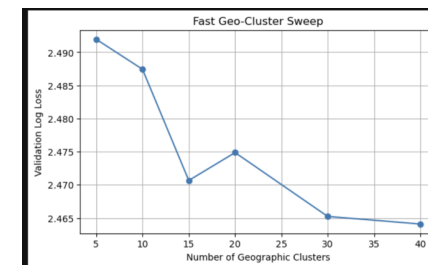


Figure #4: Geo-Cluster Sweep

As observed, validation log loss consistently improves, and had diminishing returns after around 30-40 clusters. This suggests localized crime structure. Also, for reference in this section, I optimized runtime using float32 conversion, MiniBatchKMeans, and holdout validation.

So, after preprocessing these are the predictors I was going to use for my models: Temporal; hour, weekday, month, cyclical transformations; Geographic; district, longitude/latitude, geographic cluster ID's; address-based; intersection indicator, street encodings; Intersection features; district x intersection, time-location interactions. So, I had around 81 predictors in my final logistic regression, and a smaller categorical feature set for Naïve Bayes. The resulting feature matrix combined temporal periodicity, geographic locality, and categorical location structure into a unified predictive representation used for all subsequent model training experiments.

Multinomial logistic regression was my primary predictive model due to its probabilistic interpretation, scalability, and compatibility with engineered categorical and spatial features. The model is meant to predict probability distribution across crime classes and is suitable for multiclass classification. So, each class receives a predicted probability, and the probabilities sum to 1. I also implemented regularization through C tuning. This was in an attempt to prevent overfitting, especially with my engineered predictors. Smaller c produced better generalization performance and improved validation log loss. So aggressive regularization helped stabilize predictions.

experiment		val_log_loss	val_accuracy	train_log_loss
4	C=0.1	2.450834	0.250007	2.443260
5	C=0.2	2.450913	0.250068	2.443056
6	C=0.3	2.450947	0.250022	2.443005
3	C=0.05	2.451042	0.257817	2.443681
2	C=0.03	2.451341	0.257513	2.444456
1	C=0.01	2.451578	0.257529	2.445241
0	C=0.005	2.452245	0.257538	2.446417

Figure #5: C-Tuning Experiments

All of this was to improve compatibility with my engineered featured: one-hot encoded categorical variables, cyclical predictors, geographic clusters, interaction terms. Logistic regression demonstrated substantial performance gains after incorporating cyclical temporal encoding and geographic clustering, ultimately becoming the strongest-performing model.

I then tested categorical Naïve Bayes as a simpler probabilistic baseline since it is computationally efficient and interpretable. The core assumption of NB is that the conditional independence among predictors is given in the class label. This is unrealistic for crime predictors because temporal, spatial, and district-based variables are strongly correlated. This led me to a few differences in my preprocessing: categorical/discretized inputs were required, continuous geographic information simplified, and fewer predictors used. I then tuned the alpha values, since Laplace smoothing prevents zero probabilities. In NB, feature engineering had minimal impact, geographic clustering degraded performance, and validation loss remained substantially worse than logistic regression.

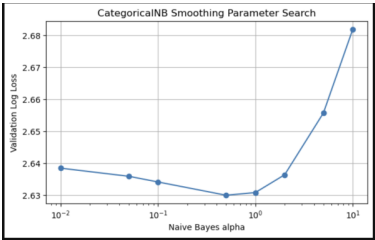


Figure #6: Alpha Value Tuning

Regarding validation I used stratified K-fold and holdout validation. I used holdout validation for all basic experimentation to optimize runtime, and then used stratified K-fold for

the optimized models to have full validation results. I employed these techniques because of the computational constraints: the data set is extremely large, geographic clustering is expensive, and I already optimized runtime earlier. For reference I used seed 12345. Regarding my evaluation metrics, I used log loss as my main metric. It evaluated predicted probabilities, penalizes overconfident incorrect predictions, and is more informative than accuracy for multiclass problems. y_i is the true class indicator, while p_i is the predicted probability.

$$-\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Accuracy was included for interpretability, but not an optimization target. In multiclass crime prediction, calibration is more informative than raw classification accuracy because many categories overlap substantially in a feature space. So, after establishing the preprocessing pipeline and modeling framework, extensive experiments were conducted to evaluate the impact of feature engineering and spatial modeling on predictive performance.

Initial baseline logistic regression models demonstrated moderate predictive performance, but struggled to capture nonlinear temporal and geographic structure. So, several feature engineering strategies were evaluated to improve probabilistic prediction quality. The baseline logistic regression of one-hot encoded categories, temporal decomposition, and distinct variables had a limited ability to capture periodicity and localized structure. Cyclical encoding had meaningful improvement in log loss (around .3) and confirmed the crime patterns exhibit strong periodic behavior over daily and weekly cycles. Regarding interaction terms, I only implemented one and that was between district and intersection. I only saw marginal gains within this aspect of the model. The next feature of my model was geographic clustering, which was my largest reduction in validation log loss, which shows spatial locality is extremely important. I

performed a geo cluster sweep, which was shown earlier in this report. The result suggests that crime patterns are highly localized geographically and benefit from spatial partitioning beyond coarse police district information.

Logistic Regression Configuration		Validation Log Loss			
Baseline		2.47+			
+ Cyclical Encoding		2.455			
+ Interaction Features		marginal improvement			
+ Geographic Clustering		2.45 (best)			

	model	test_log_loss	test_accuracy	train_log_loss	n_predictors
0	Optimized Logistic	2.463315	0.249857	2.458546	81
1	Optimized Naive Bayes	2.625005	0.236819	2.615698	16

Logistic regression substantially outperformed Naïve bayes because crime features exhibit strong dependency structures across time, district, and geographic location. Ther weaker NB performance was expected due to the strong correlation structure within the predictor set. Below are plots visualizing all temporal and geographic aspects of this dataset that weren’t already mentioned.

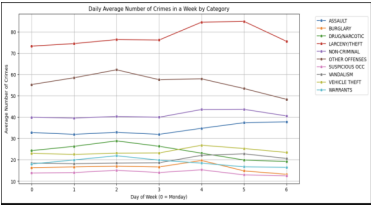


Figure #7: Daily Average in a Week

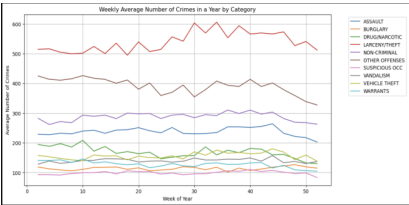


Figure #8: Weekly Average in a Year

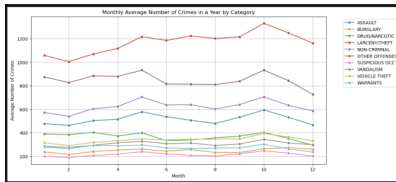


Figure #9: Monthly Average by Year

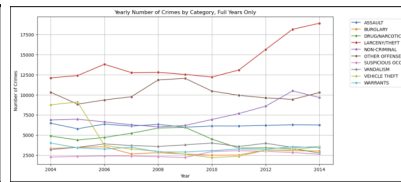


Figure #10: Yearly Average

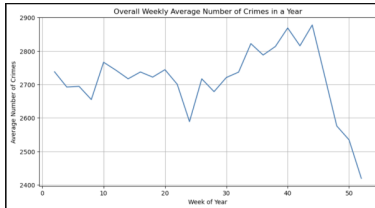


Figure #11: Weekly Averages Summed

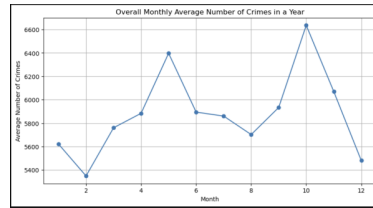


Figure #12: Monthly Averages Summed

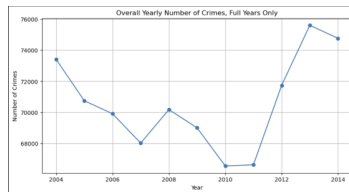


Figure #13: Yearly Averages Summed

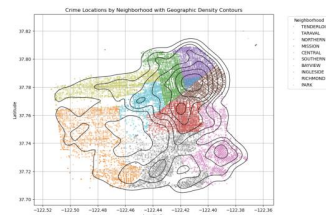


Figure #14: Crime Locations by Neighborhood with Geographic Density Contours

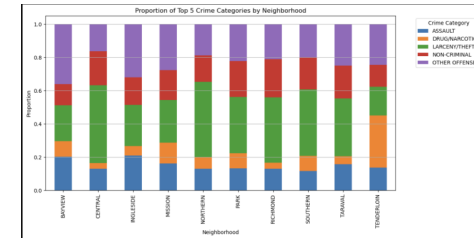


Figure #15: Proportions of Top 5 Crime Categories by Neighborhood

The SF crime dataset is a nonlinear, large dataset, which is generally difficult to work with. My goal was to try to create a multiclass SF crime prediction model. The strongest improvement came from geographic clustering, then cyclical encoding. Features like interaction terms provided marginal gains. The best overall model was my Logistic regression model, while my Naïve Bayes was weaker due to its independence assumptions. This project demonstrated that effective crime prediction depends less on model complexity, and more on carefully engineered spatial-temporal representations.

SF Crime Project

1. Project Overview

This project builds a multiclass probabilistic model to predict San Francisco crime categories from temporal, geographic, district, and address-based features. The primary evaluation metric is multiclass log loss because the goal is to produce useful probability estimates, not only hard-label predictions.

2. Imports and Configuration

```
In [100]: import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib import pyplot as plt
import datetime as dt
from sklearn.model_selection import train_test_split, RepeatedKFold
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder, OrdinalEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import log_loss
from patsy import dmatrix
from sklearn.metrics import (log_loss, accuracy_score)
from sklearn.model_selection import StratifiedKFold
from sklearn.cluster import MiniBatchKMeans
```

3. Data Loading and Cleaning

This section loads the raw San Francisco crime dataset, standardizes column names, removes leakage variables, filters unreliable geographic observations, and parses timestamps and addresses into structured predictors.

sf_crime function

```
In [101]: def sf_crime(full=False,
                    train=True,
                    split_seed=12345,
                    encoding=False,
                    datetime_index=True):

    # File path for the San Francisco crime training dataset
    path_to_file = '/Users/aoive/Downloads/san_fransisco_crime_train.csv'

    # Load the dataset
```

```
X = pd.read_csv(path_to_file, header=0)

# Some data cleansing

# strip whitespace and make everything lowercase.
X.columns = X.columns.str.strip().str.lower()

# Rename dates to date_time so the column name is clearer
X = X.rename(columns={'dates': 'date_time'})

# Drop columns we are not allowed / not planning to use as predictors
X = X.drop(columns=['descript', 'resolution'])

# Clean string columns by removing leading/trailing whitespace
fun_dct = {
    'category': lambda x: x.strip(),
    'pddistrict': lambda x: x.strip()
}
X[['category', 'pddistrict']] = X.transform(fun_dct)

slicing = X['address'].str.contains('/', na=False) & (~X['address'].str.contains(
X.loc[slicing, 'address'] = (
    X.loc[slicing, 'address']
    .str.replace('/', ' / ', regex=False)
    .str.replace(r'\s+', ' ', regex=True)
    .str.strip()
)

# Discard unreliable geographic observations
X = X.loc[X['x'] < -122, :].copy()

# Feature engineering

# Convert date_time to an actual datetime object.\
X['date_time'] = pd.to_datetime(X['date_time'])

# Extract standard time features.\
X['year'] = X.date_time.dt.year
X['month'] = X.date_time.dt.month
X['day'] = X.date_time.dt.day
X['hour'] = X.date_time.dt.hour
X['minute'] = X.date_time.dt.minute
X['second'] = X.date_time.dt.second

# Numeric timestamp measured in seconds from the first dataset time.\
X['dt_numeric'] = (
    X.date_time - pd.to_datetime('2003-01-06 00:01:00')
).dt.total_seconds()

# More datetime-derived features.\
X['day_of_week'] = X.date_time.dt.weekday
X['days_in_month'] = X.date_time.dt.days_in_month
X['day_of_year'] = X.date_time.dt.day_of_year
X['week_of_year'] = X.date_time.dt.isocalendar().week

#Parsing address
```

```

# intersection = 1 if the address contains " / "
# intersection = 0 if the address is a block address
X['intersection'] = X['address'].str.contains(' / ', regex=False).astype('int8')

# Split address into first and second street components
address_split = X['address'].str.split(' / ', expand=True)

# First address component
X['street_1'] = address_split[0].str.strip()

# Second address component
X['street_2'] = address_split[1].str.strip()

# Extract block number from block-style addresses
X['block'] = X['street_1'].str.extract(r'^(\d+)\s+Block of', expand=False)

# Convert block to numeric
X['block'] = pd.to_numeric(X['block'], errors='coerce')

# Remove the block prefix from street_1
X['street_1'] = (
    X['street_1']
    .str.replace(r'^\d+\s+Block of\s+', '', regex=True)
    .str.strip()
)

# For intersections, street_2 remains the second street
X.loc[X['intersection'] == 0, 'street_2'] = None

# Encoding street_1 and street_2 only

if encoding == True:

    # Build a dictionary using all unique street names appearing in either street
    street_values = (
        pd.concat([X['street_1'], X['street_2']], axis=0)
        .dropna()
        .drop_duplicates()
        .sort_values()
    )

    # Dictionary: street name -> integer code
    street_dictionary = {
        street: code for code, street in enumerate(street_values)
    }

    # Map street names to integer codes
    X['street_1'] = X['street_1'].map(street_dictionary)
    X['street_2'] = X['street_2'].map(street_dictionary)

    # Non-intersection rows have no second street, so encode missing street_2 as
    X['street_2'] = X['street_2'].fillna(-1)

    # Intersections have no block number, so encode missing block as 0
    X['block'] = X['block'].fillna(0)

```

```

# Convert encoded street/block columns to integer type
X['street_1'] = X['street_1'].astype('int16')
X['street_2'] = X['street_2'].astype('int16')
X['block'] = X['block'].astype('int16')

#type control / cleanup

# Smaller integer types save memory on a very large dataset
features = [
    'minute', 'second', 'hour', 'day', 'month',
    'day_of_week', 'days_in_month', 'week_of_year',
    'intersection'
]
X[features] = X[features].astype('int8')

# These are larger time features.
X[['day_of_year', 'year']] = X[['day_of_year', 'year']].astype('int16')

zero_variance_cols = [
    col for col in X.columns
    if col != 'category' and X[col].nunique(dropna=False) <= 1
]

if len(zero_variance_cols) > 0:
    X = X.drop(columns=zero_variance_cols)

# Set date_time as row index
if datetime_index == True:
    X = X.set_index(keys='date_time', drop=False)
    X.index.name = 'key1'

# Drop columns that are redundant after feature engineering
cols_to_drop = ['dayofweek', 'days_in_month', 'second']
X = X.drop(columns=cols_to_drop, errors='ignore')

# If full=True, return the entire processed dataset
if full == True:
    X = X.sort_values(by=['date_time'])
    return X

# Otherwise split into train/test sets using stratification by category
else:
    y = X['category']

    X_train, X_test = train_test_split(
        X,
        test_size=.4,
        random_state=split_seed,
        stratify=y
    )

    if train == True:
        X_train = X_train.sort_values(by=['date_time'])
        return X_train

```

```

else:
    X_test = X_test.sort_values(by=['date_time'])
    return X_test

```

4. Exploratory Data Analysis

```

In [102]: # Test encoded version
X_train_encoded = sf_crime(train=True, encoding=True, datetime_index=True)

X_train_encoded[['address', 'block', 'street_1', 'street_2', 'intersection']].head(16)

```

```

Out[102]:

```

	address	block	street_1	street_2	intersection
key1					
2003-01-06 00:01:00	1800 Block of NEWCOMB AV	1800	1348	-1	0
2003-01-06 00:15:00	FARALLONES ST / CAPITOL AV	0	701	345	1
2003-01-06 00:20:00	1500 Block of SHAFTER AV	1500	1734	-1	0
2003-01-06 00:20:00	1500 Block of SHAFTER AV	1500	1734	-1	0
2003-01-06 00:31:00	CLEMENT ST / 14TH AV	0	433	7	1
2003-01-06 00:33:00	2800 Block of FOLSOM ST	2800	728	-1	0
2003-01-06 00:40:00	POLK ST / CALIFORNIA ST	0	1483	334	1
2003-01-06 00:42:00	TAYLOR ST / GEARY ST	0	1852	778	1
2003-01-06 00:55:00	1300 Block of WEBSTER ST	1300	2002	-1	0
2003-01-06 00:55:00	1300 Block of WEBSTER ST	1300	2002	-1	0

```

In [103]: # Check types and missingness.
X_train_encoded[['block', 'street_1', 'street_2', 'intersection']].info()

<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 526789 entries, 2003-01-06 00:01:00 to 2015-05-13 23:33:00
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    block      526789 non-null  int16
1   street_1   526789 non-null  int16
2   street_2   526789 non-null  int16
3   intersection 526789 non-null  int8
dtypes: int16(3), int8(1)
memory usage: 7.5 MB

```

5. Feature Engineering and Preprocessing

This section converts raw temporal, spatial, and categorical fields into model-ready predictors. The main engineered features include cyclical time encodings, intersection indicators, street

encodings, police-district indicators, geographic clusters, and selected interaction terms.

Data preprocessing function

```

In [104]: def data_preprocessing(X_train,
                                X_test,
                                y_train,
                                y_test,
                                selected_predictors=[],
                                one_hot_encoding=False,
                                drop_first=False,
                                full_preprocessing=True,
                                use_interaction=False,
                                use_pca=False,
                                pcs=[],
                                dataframe_format=False,
                                cyclical_encoding=False,
                                use_geo_cluster=False,
                                n_geo_clusters=40,
                                use_geo_hour=False,
                                use_geo_day=False,
                                use_district_hour=False,
                                use_float32=True):

    # Store original indexing
    X_train_index = X_train.index
    X_test_index = X_test.index

    # Encode output Labels
    label_enc = LabelEncoder()
    label_enc.fit(y_train)

    # Transform category Labels into integer class Labels
    y_train = label_enc.transform(y_train)
    y_test = label_enc.transform(y_test)

    # int8 saves memory because there are only around 39 classes
    y_train = y_train.astype('int8')
    y_test = y_test.astype('int8')

    # Dropping predictors that should not enter the model
    slicing = ['category', 'date_time', 'address']

    X_train = X_train.drop(columns=slicing, errors='ignore')
    X_test = X_test.drop(columns=slicing, errors='ignore')

    #Block feature rescaling
    X_train['block'] = np.where(X_train['block'].notna(), X_train['block'] / 100 + 1,
                                X_test['block'] = np.where(X_test['block'].notna(), X_test['block'] / 100 + 1, 0)

    #Optional cyclical encoding
    if cyclical_encoding == True:

```



```

# Hour cycle: 24 hours
X_train['hour_sin'] = np.sin(2 * np.pi * X_train['hour'] / 24)
X_train['hour_cos'] = np.cos(2 * np.pi * X_train['hour'] / 24)
X_test['hour_sin'] = np.sin(2 * np.pi * X_test['hour'] / 24)
X_test['hour_cos'] = np.cos(2 * np.pi * X_test['hour'] / 24)

# Day-of-week cycle: 7 days
X_train['day_of_week_sin'] = np.sin(2 * np.pi * X_train['day_of_week'] / 7)
X_train['day_of_week_cos'] = np.cos(2 * np.pi * X_train['day_of_week'] / 7)
X_test['day_of_week_sin'] = np.sin(2 * np.pi * X_test['day_of_week'] / 7)
X_test['day_of_week_cos'] = np.cos(2 * np.pi * X_test['day_of_week'] / 7)

# Month cycle: 12 months
X_train['month_sin'] = np.sin(2 * np.pi * X_train['month'] / 12)
X_train['month_cos'] = np.cos(2 * np.pi * X_train['month'] / 12)
X_test['month_sin'] = np.sin(2 * np.pi * X_test['month'] / 12)
X_test['month_cos'] = np.cos(2 * np.pi * X_test['month'] / 12)

# Optional spatial clustering feature
if use_geo_cluster == True:

    # Fit MiniBatchKMeans only on training coordinates
    geo_kmeans = MiniBatchKMeans(
        n_clusters=n_geo_clusters,
        random_state=12345,
        batch_size=2048,
        n_init=1,
        max_iter=50
    )
    # Fit clusters using train x/y
    geo_kmeans.fit(X_train[['x', 'y']])

    # Assign each observation to a geographic cluster
    X_train['geo_cluster'] = geo_kmeans.predict(X_train[['x', 'y']])
    X_test['geo_cluster'] = geo_kmeans.predict(X_test[['x', 'y']])

    # Store as small integer type
    X_train['geo_cluster'] = X_train['geo_cluster'].astype('int16')
    X_test['geo_cluster'] = X_test['geo_cluster'].astype('int16')

# Optional targeted spatial-temporal interaction features
if use_geo_cluster == True and use_geo_hour == True:

    X_train["geo_hour"] = (
        X_train["geo_cluster"].astype(str)
        + "_"
        + X_train["hour"].astype(str)
    )

    X_test["geo_hour"] = (
        X_test["geo_cluster"].astype(str)
        + "_"
        + X_test["hour"].astype(str)
    )

```

```

if use_geo_cluster == True and use_geo_day == True:

    X_train["geo_day"] = (
        X_train["geo_cluster"].astype(str)
        + "_"
        + X_train["day_of_week"].astype(str)
    )

    X_test["geo_day"] = (
        X_test["geo_cluster"].astype(str)
        + "_"
        + X_test["day_of_week"].astype(str)
    )

if use_district_hour == True:

    X_train["district_hour"] = (
        X_train["pddistrict"].astype(str)
        + "_"
        + X_train["hour"].astype(str)
    )

    X_test["district_hour"] = (
        X_test["pddistrict"].astype(str)
        + "_"
        + X_test["hour"].astype(str)
    )

#Encode categorical predictors
from warnings import filterwarnings
filterwarnings("ignore", message=".*These unknown categories will be encoded as a")

if one_hot_encoding == True:
    categorical_enc = OneHotEncoder(
        sparse_output=False,
        handle_unknown='ignore',
        drop='first' if drop_first == True else None,
        dtype='int8'
    )
else:
    categorical_enc = OrdinalEncoder(
        handle_unknown='use_encoded_value',
        unknown_value=np.nan,
        dtype='float64'
    )

# Categorical encoding
# pddistrict is categorical.
# geo_cluster is also categorical if we created it.
categorical_cols = ['pddistrict']

if use_geo_cluster == True:
    categorical_cols.append('geo_cluster')

if use_geo_hour == True:
    categorical_cols.append('geo_hour')

```

```

if use_geo_day == True:
    categorical_cols.append('geo_day')

if use_district_hour == True:
    categorical_cols.append('district_hour')

categorical_enc.fit(X_train[categorical_cols])

encoded_names = categorical_enc.get_feature_names_out(categorical_cols)

X_train[encoded_names] = categorical_enc.transform(X_train[categorical_cols])
X_test[encoded_names] = categorical_enc.transform(X_test[categorical_cols])

if one_hot_encoding == True:
    X_train = X_train.drop(columns=categorical_cols)
    X_test = X_test.drop(columns=categorical_cols)

if one_hot_encoding == True:
    X_train = X_train.drop(columns=categorical_cols, errors='ignore')
    X_test = X_test.drop(columns=categorical_cols, errors='ignore')

# Required interaction terms: pddistrict x intersection
if use_interaction == True:

    # Find all district-encoded columns
    district_cols = [
        col for col in X_train.columns
        if col.startswith('pddistrict')
    ]

    for col in district_cols:

        # Name interaction as district_col:intersection
        interaction_name = col + ':intersection'

        # Multiply district indicator/code by intersection dummy
        X_train[interaction_name] = X_train[col] * X_train['intersection']
        X_test[interaction_name] = X_test[col] * X_test['intersection']

# Remove possible zero-variance predictors
zero_var_cols = [
    col for col in X_train.columns
    if X_train[col].nunique(dropna=False) <= 1
]

if len(zero_var_cols) > 0:
    X_train = X_train.drop(columns=zero_var_cols)
    X_test = X_test.drop(columns=zero_var_cols, errors='ignore')

# Feature selection

all_predictors = X_train.columns

if use_interaction == True:

```

```

# Directly selected predictors.
slicing_1 = all_predictors.isin(selected_predictors)

# If the user selects "pddistrict", keep all one-hot pddistrict columns.
slicing_2 = np.zeros(len(all_predictors), dtype=bool)
if 'pddistrict' in selected_predictors:
    slicing_2 = slicing_2 | all_predictors.str.startswith('pddistrict')

# If the user selects "intersection", keep intersection and district inte
slicing_3 = np.zeros(len(all_predictors), dtype=bool)
if 'intersection' in selected_predictors:
    slicing_3 = (
        slicing_3
        | (all_predictors == 'intersection')
        | all_predictors.str.contains(':intersection', regex=False)
    )

slicing_4 = all_predictors.isin(selected_predictors)

slicing = slicing_1 | slicing_2 | slicing_3 | slicing_4

else:
    slicing = all_predictors.isin(selected_predictors)

if 'pddistrict' in selected_predictors:
    slicing = slicing | all_predictors.str.startswith('pddistrict')

# Avoid accidentally producing an empty dataframe
if X_train.loc[:, slicing].shape[1] > 0:
    X_train = X_train.loc[:, slicing]
    X_test = X_test.loc[:, slicing]

# Save predictor names before scaling/PCA
predictors = X_train.columns.to_list()

# Full preprocessing: scaling and optional PCA
if full_preprocessing == True:

    scaler = StandardScaler()

    scaler.fit(X_train)

# Transform both train and test using the training scaler
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
if use_float32 == True:
    X_train = X_train.astype(np.float32)
    X_test = X_test.astype(np.float32)

# Optional PCA
if use_pca == True:

    # Check whether a valid list of PCs was given
    condition = (pcs != []) and ((np.array(pcs + [0]).max()) <= len(predictor

    # If pcs=[0,1,2], fit PCA with 3 components

```

```

# If pcs is empty, fit PCA with default n_components=None
nbr_pcs = np.array(pcs).max() + 1 if condition == True else None

pca = PCA(n_components=nbr_pcs)

# Fit PCA only on training data
pca.fit(X_train)

# Transform train/test
X_train = pca.transform(X_train)
X_test = pca.transform(X_test)

# Predictor names become component indices
predictors = np.arange(0, X_train.shape[1]).tolist()

# If specific PCs are requested, keep only those
if condition == True:
    X_train = X_train[:, pcs]
    X_test = X_test[:, pcs]
    predictors = pcs

# 10. Return dataframe format if requested
if dataframe_format == True:
    X_train = pd.DataFrame(X_train, columns=predictors, index=X_train_index)
    X_test = pd.DataFrame(X_test, columns=predictors, index=X_test_index)

return X_train, X_test, y_train, y_test, predictors, label_enc

```

```

In [105]: # Test #1: Base preprocessing, ordinal district, no interactions
# Load train/test from sf_crime
X_train = sf_crime(train=True, encoding=True, datetime_index=True)
X_test = sf_crime(train=False, encoding=True, datetime_index=True)

# Separate response variable
y_train = X_train['category']
y_test = X_test['category']

# Run simplest preprocessing
Xtr_base, Xte_base, ytr_base, yte_base, predictors_base, label_base = data_preprocessing(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    selected_predictors=[],
    one_hot_encoding=False,
    full_preprocessing=True,
    use_interaction=False,
    dataframe_format=True
)

print("Base predictors:")
print(predictors_base)

print("\nTrain shape:", Xtr_base.shape)
print("Test shape:", Xte_base.shape)

```

```
Xtr_base.head()
```

Base predictors:

```
['pddistrict', 'x', 'y', 'year', 'month', 'day', 'hour', 'minute', 'dt_numeric', 'day_of_week', 'day_of_year', 'week_of_year', 'intersection', 'street_1', 'street_2', 'block']
```

Train shape: (526789, 16)

Test shape: (351193, 16)

```
Out[105]:
```

	pddistrict	x	y	year	month	day	hour	minute
key1								
2003-01-06 00:01:00	-1.551722	1.101191	-1.193149	-1.570463	-1.584489	-1.088246	-2.04476	-1.0305
2003-01-06 00:15:00	1.241600	-1.435518	-2.192628	-1.570463	-1.584489	-1.088246	-2.04476	-0.2772
2003-01-06 00:20:00	-1.551722	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.0082
2003-01-06 00:20:00	-1.551722	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.0082
2003-01-06 00:31:00	0.543269	-1.987584	0.641552	-1.570463	-1.584489	-1.088246	-2.04476	0.5836

```

In [106]: # Test #2: One-hot district, no interactions
Xtr_oh, Xte_oh, ytr_oh, yte_oh, predictors_oh, label_oh = data_preprocessing(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    selected_predictors=[],
    one_hot_encoding=True,
    drop_first=False,
    full_preprocessing=True,
    use_interaction=False,
    dataframe_format=True
)

print("One-hot predictors:")
print(predictors_oh)

print("\nTrain shape:", Xtr_oh.shape)
print("Test shape:", Xte_oh.shape)

Xtr_oh.head()

```

```
One-hot predictors:
['x', 'y', 'year', 'month', 'day', 'hour', 'minute', 'dt_numeric', 'day_of_week', 'day_of_year', 'week_of_year', 'intersection', 'street_1', 'street_2', 'block', 'pddistrict_BAYVIEW', 'pddistrict_CENTRAL', 'pddistrict_INGLESIDE', 'pddistrict_MISSION', 'pddistrict_NORTHERN', 'pddistrict_PARK', 'pddistrict_RICHMOND', 'pddistrict_SOUTHERN', 'pddistrict_TARAVAL', 'pddistrict_TENDERLOIN']
```

```
Train shape: (526789, 25)
Test shape: (351193, 25)
```

Out[106]:

	x	y	year	month	day	hour	minute	dt_num
key1								
2003-01-06 00:01:00	1.101191	-1.193149	-1.570463	-1.584489	-1.088246	-2.04476	-1.030512	-1.703
2003-01-06 00:15:00	-1.435518	-2.192628	-1.570463	-1.584489	-1.088246	-2.04476	-0.277252	-1.703
2003-01-06 00:20:00	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.008231	-1.703
2003-01-06 00:20:00	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.008231	-1.703
2003-01-06 00:31:00	-1.987584	0.641552	-1.570463	-1.584489	-1.088246	-2.04476	0.583615	-1.703

5 rows x 25 columns

```
In [107]: #Test 3: One-hot district w/ interactions
Xtr_int, Xte_int, ytr_int, yte_int, predictors_int, label_int = data_preprocessing(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    selected_predictors=[],
    one_hot_encoding=True,
    drop_first=False,
    full_preprocessing=True,
    use_interaction=True,
    dataframe_format=True
)

print("Interaction predictors:")
print([col for col in predictors_int if ':intersection' in str(col)])

print("\nTrain shape:", Xtr_int.shape)
print("Test shape:", Xte_int.shape)
```

```
Xtr_int.head()
```

```
Interaction predictors:
['pddistrict_BAYVIEW:intersection', 'pddistrict_CENTRAL:intersection', 'pddistrict_INGLESIDE:intersection', 'pddistrict_MISSION:intersection', 'pddistrict_NORTHERN:intersection', 'pddistrict_PARK:intersection', 'pddistrict_RICHMOND:intersection', 'pddistrict_SOUTHERN:intersection', 'pddistrict_TARAVAL:intersection', 'pddistrict_TENDERLOIN:intersection']
```

```
Train shape: (526789, 35)
Test shape: (351193, 35)
```

Out[107]:

	x	y	year	month	day	hour	minute	dt_num
key1								
2003-01-06 00:01:00	1.101191	-1.193149	-1.570463	-1.584489	-1.088246	-2.04476	-1.030512	-1.703
2003-01-06 00:15:00	-1.435518	-2.192628	-1.570463	-1.584489	-1.088246	-2.04476	-0.277252	-1.703
2003-01-06 00:20:00	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.008231	-1.703
2003-01-06 00:20:00	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.008231	-1.703
2003-01-06 00:31:00	-1.987584	0.641552	-1.570463	-1.584489	-1.088246	-2.04476	0.583615	-1.703

5 rows x 35 columns

```
In [108]: # Test #4: Cyclical encoding
Xtr_cyc, Xte_cyc, ytr_cyc, yte_cyc, predictors_cyc, label_cyc = data_preprocessing(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    selected_predictors=[],
    one_hot_encoding=True,
    drop_first=False,
    full_preprocessing=True,
    use_interaction=True,
    cyclical_encoding=True,
    dataframe_format=True
)

print("Cyclical predictors:")
print([col for col in predictors_cyc if 'sin' in str(col) or 'cos' in str(col)])
```

```
print("\nTrain shape:", Xtr_cyc.shape)
print("Test shape:", Xte_cyc.shape)
```

```
Xtr_cyc.head()
```

Cyclical predictors:

```
['hour_sin', 'hour_cos', 'day_of_week_sin', 'day_of_week_cos', 'month_sin', 'month_cos']
```

Train shape: (526789, 41)

Test shape: (351193, 41)

Out[108]:

	x	y	year	month	day	hour	minute	dt_num
key1								

2003-01-06 00:01:00	1.101191	-1.193149	-1.570463	-1.584489	-1.088246	-2.04476	-1.030512	-1.703
---------------------	----------	-----------	-----------	-----------	-----------	----------	-----------	--------

2003-01-06 00:15:00	-1.435518	-2.192628	-1.570463	-1.584489	-1.088246	-2.04476	-0.277252	-1.703
---------------------	-----------	-----------	-----------	-----------	-----------	----------	-----------	--------

2003-01-06 00:20:00	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.008231	-1.703
---------------------	----------	-----------	-----------	-----------	-----------	----------	-----------	--------

2003-01-06 00:20:00	1.305241	-1.509596	-1.570463	-1.584489	-1.088246	-2.04476	-0.008231	-1.703
---------------------	----------	-----------	-----------	-----------	-----------	----------	-----------	--------

2003-01-06 00:31:00	-1.987584	0.641552	-1.570463	-1.584489	-1.088246	-2.04476	0.583615	-1.703
---------------------	-----------	----------	-----------	-----------	-----------	----------	----------	--------

5 rows x 41 columns

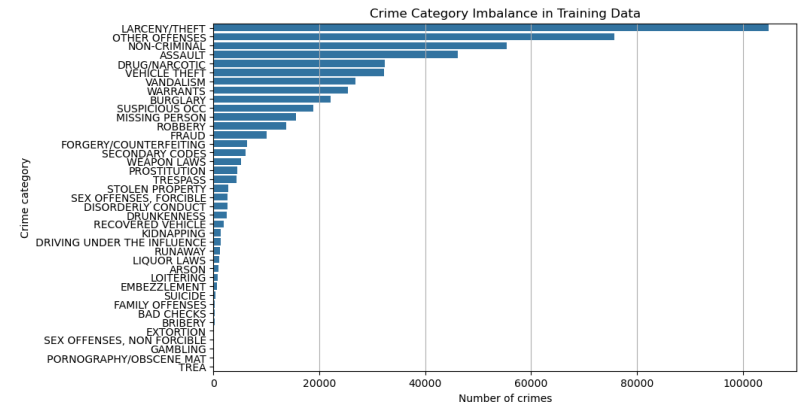
In [109]:

```
plt.figure(figsize=(10, 6))

category_counts = X_train['category'].value_counts()

sns.barplot(
    x=category_counts.values,
    y=category_counts.index
)

plt.xlabel("Number of crimes")
plt.ylabel("Crime category")
plt.title("Crime Category Imbalance in Training Data")
plt.grid(axis='x')
plt.show()
```



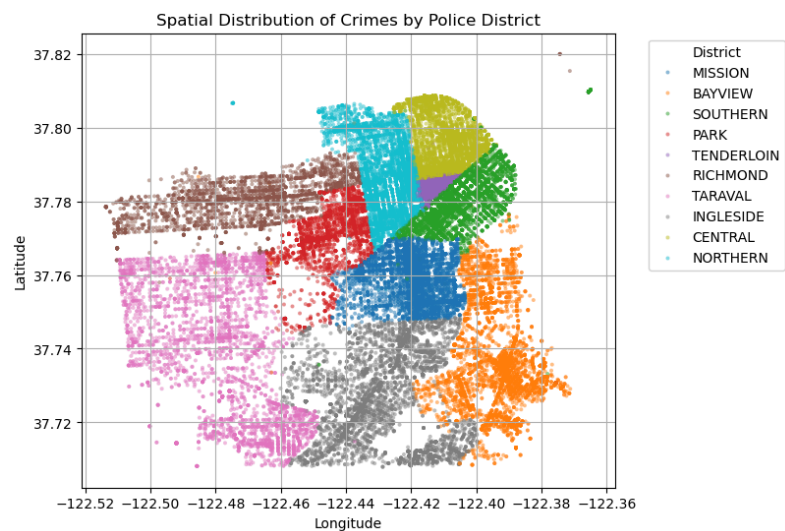
In [110]:

```
plt.figure(figsize=(7, 6))

sample_df = X_train.sample(n=min(50000, len(X_train)), random_state=12345)

sns.scatterplot(
    data=sample_df,
    x='x',
    y='y',
    hue='pddistrict',
    s=8,
    alpha=0.5,
    linewidth=0
)

plt.title("Spatial Distribution of Crimes by Police District")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend(title="District", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid()
plt.show()
```

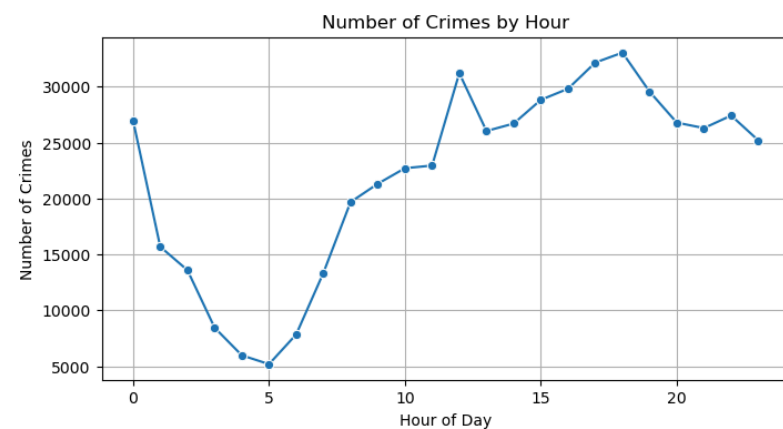


```
In [111]: plt.figure(figsize=(8, 4))

hour_df = (
    X_train
    .groupby('hour')
    .agg(nbr_crimes=('category', 'count'))
    .reset_index()
)

sns.lineplot(
    data=hour_df,
    x='hour',
    y='nbr_crimes',
    marker='o'
)

plt.title("Number of Crimes by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Crimes")
plt.grid()
plt.show()
```

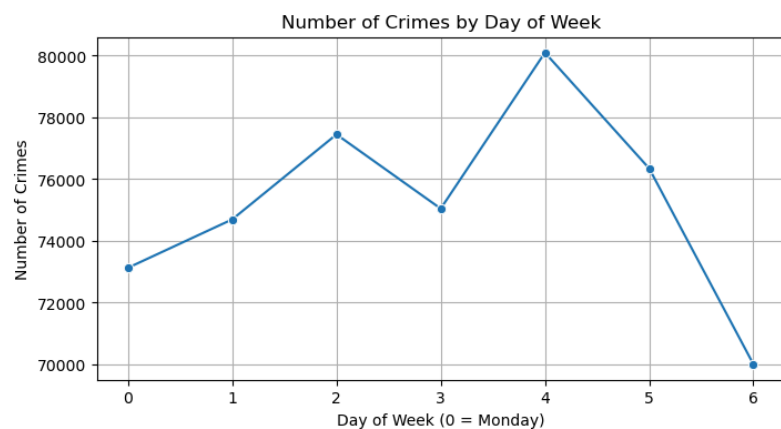


```
In [112]: plt.figure(figsize=(8, 4))

dow_df = (
    X_train
    .groupby('day_of_week')
    .agg(nbr_crimes=('category', 'count'))
    .reset_index()
)

sns.lineplot(
    data=dow_df,
    x='day_of_week',
    y='nbr_crimes',
    marker='o'
)

plt.title("Number of Crimes by Day of Week")
plt.xlabel("Day of Week (0 = Monday)")
plt.ylabel("Number of Crimes")
plt.grid()
plt.show()
```

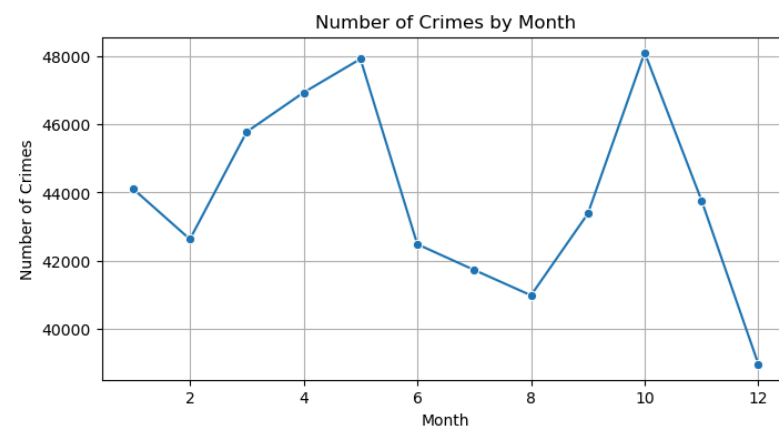


```
In [113]: plt.figure(figsize=(8, 4))

month_df = (
    X_train
    .groupby('month')
    .agg(nbr_crimes=('category', 'count'))
    .reset_index()
)

sns.lineplot(
    data=month_df,
    x='month',
    y='nbr_crimes',
    marker='o'
)

plt.title("Number of Crimes by Month")
plt.xlabel("Month")
plt.ylabel("Number of Crimes")
plt.grid()
plt.show()
```



6. Modeling Functions

```
In [114]: def predictive_algo(X_train,
                             X_test,
                             y_train,
                             y_test,
                             model_type="logistic",
                             selected_predictors=[],
                             one_hot_encoding=False,
                             drop_first=False,
                             full_preprocessing=True,
                             use_interaction=False,
                             use_pca=False,
                             pcs=[],
                             dataframe_format=False,
                             cyclical_encoding=False,
                             use_geo_cluster=False,
                             n_geo_clusters=20,
                             use_geo_hour=False,
                             use_geo_day=False,
                             use_district_hour=False,
                             use_float32=True,
                             C=1.0,
                             max_iter=500,
                             alpha=1.0):

    # 1. Choose model
    if model_type == "logistic":
        model = LogisticRegression(
            C=C,
            max_iter=max_iter,
            solver="lbfgs"
        )
```

```

elif model_type == "nb":
    model = CategoricalNB(alpha=alpha)

    # Force NB-safe preprocessing choices
    full_preprocessing = False
    one_hot_encoding = False
    use_pca = False

else:
    raise ValueError("model_type must be either 'logistic' or 'nb'.")

# 2. Create X_train_prep and X_test_prep
X_train_prep, X_test_prep, y_train_enc, y_test_enc, predictors, label_enc = data
X_train=X_train,
X_test=X_test,
y_train=y_train,
y_test=y_test,
selected_predictors=selected_predictors,
one_hot_encoding=one_hot_encoding,
drop_first=drop_first,
full_preprocessing=full_preprocessing,
use_geo_cluster=use_geo_cluster,
n_geo_clusters=n_geo_clusters,
use_geo_hour=use_geo_hour,
use_geo_day=use_geo_day,
use_district_hour=use_district_hour,
use_float32=use_float32,
use_interaction=use_interaction,
use_pca=use_pca,
pcs=pcs,
dataframe_format=dataframe_format,
cyclical_encoding=cyclical_encoding
)

# 3. Do NB-specific cleanup
if model_type == "nb":

    if not isinstance(X_train_prep, pd.DataFrame):
        X_train_prep = pd.DataFrame(X_train_prep, columns=predictors)
        X_test_prep = pd.DataFrame(X_test_prep, columns=predictors)

    for col in X_train_prep.columns:

        X_train_prep[col] = X_train_prep[col].fillna(-1)
        X_test_prep[col] = X_test_prep[col].fillna(-1)

        if X_train_prep[col].nunique() > 30:

            train_bins, bin_edges = pd.qcut(
                X_train_prep[col],
                q=10,
                retbins=True,
                labels=False,
                duplicates="drop"
            )

```

```

            test_bins = pd.cut(
                X_test_prep[col],
                bins=bin_edges,
                labels=False,
                include_lowest=True
            )

            X_train_prep[col] = train_bins
            X_test_prep[col] = test_bins

        X_train_prep[col] = X_train_prep[col].fillna(0)
        X_test_prep[col] = X_test_prep[col].fillna(0)

    min_val = min(X_train_prep[col].min(), X_test_prep[col].min())

    if min_val < 0:
        X_train_prep[col] = X_train_prep[col] - min_val
        X_test_prep[col] = X_test_prep[col] - min_val

    X_train_prep[col] = X_train_prep[col].astype("int64")
    X_test_prep[col] = X_test_prep[col].astype("int64")

# 4. Fit model
model.fit(X_train_prep, y_train_enc)

# 5. Predict probabilities
y_train_proba = model.predict_proba(X_train_prep)
y_test_proba = model.predict_proba(X_test_prep)

# 6. Predict classes
y_train_pred = model.predict(X_train_prep)
y_test_pred = model.predict(X_test_prep)

# 7. Metrics
labels = np.arange(len(label_enc.classes_))

train_log_loss = log_loss(y_train_enc, y_train_proba, labels=labels)
test_log_loss = log_loss(y_test_enc, y_test_proba, labels=labels)

train_accuracy = accuracy_score(y_train_enc, y_train_pred)
test_accuracy = accuracy_score(y_test_enc, y_test_pred)

return {
    "model_type": model_type,
    "model": model,
    "predictors": predictors,
    "label_encoder": label_enc,
    "train_log_loss": train_log_loss,
    "test_log_loss": test_log_loss,
    "train_accuracy": train_accuracy,
    "test_accuracy": test_accuracy,
    "y_train_proba": y_train_proba,
    "y_test_proba": y_test_proba,
    "y_train_pred": y_train_pred,
    "y_test_pred": y_test_pred
}

```


7. Validation Framework

Holdout validation is used for fast experimentation, while K-fold validation is reserved for final candidate models because the dataset is large and repeated fitting is computationally expensive.

```
In [119]: def model_validation(
    X,
    y,
    model_type="logistic",
    validation_type="holdout", # "holdout" or "kfold"
    selected_predictors=None,
    one_hot_encoding=False,
    drop_first=False,
    full_preprocessing=True,
    use_interaction=False,
    use_pca=False,
    pcs=None,
    cyclical_encoding=False,
    use_geo_cluster=False,
    n_geo_clusters=20,
    use_geo_hour=False,
    use_geo_day=False,
    use_district_hour=False,
    use_float32=True,
    C=1.0,
    max_iter=500,
    alpha=1.0,
    test_size=0.25,
    n_splits=4,
    split_seed=12345,
    verbose=True
):
    """
    Validate a model using either:
    - stratified holdout validation for fast experimentation
    - stratified K-fold validation for final model confirmation

    Returns:
    - summary: one-row dictionary of averaged metrics/settings
    - fold_results: dataframe of validation results by split/fold
    """

    if selected_predictors is None:
        selected_predictors = []

    if pcs is None:
        pcs = []

    if validation_type not in ["holdout", "kfold"]:
        raise ValueError("validation_type must be either 'holdout' or 'kfold'.")

    fold_results = []
```

```
# Create validation splits
if validation_type == "holdout":

    X_train_fold, X_val_fold, y_train_fold, y_val_fold = train_test_split(
        X,
        y,
        test_size=test_size,
        random_state=split_seed,
        stratify=y
    )

    splits = [(1, X_train_fold, X_val_fold, y_train_fold, y_val_fold)]

else:

    skf = StratifiedKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=split_seed
    )

    splits = []

    for fold, (train_idx, val_idx) in enumerate(skf.split(X, y), start=1):

        X_train_fold = X.iloc[train_idx].copy()
        X_val_fold = X.iloc[val_idx].copy()

        y_train_fold = y.iloc[train_idx].copy()
        y_val_fold = y.iloc[val_idx].copy()

        splits.append((fold, X_train_fold, X_val_fold, y_train_fold, y_val_fold))

# Run model on each split
for fold, X_train_fold, X_val_fold, y_train_fold, y_val_fold in splits:

    result = predictive_algo(
        X_train=X_train_fold,
        X_test=X_val_fold,
        y_train=y_train_fold,
        y_test=y_val_fold,
        model_type=model_type,
        selected_predictors=selected_predictors,
        one_hot_encoding=one_hot_encoding,
        drop_first=drop_first,
        full_preprocessing=full_preprocessing,
        use_interaction=use_interaction,
        use_pca=use_pca,
        pcs=pcs,
        dataframe_format=False,
        cyclical_encoding=cyclical_encoding,
        use_geo_cluster=use_geo_cluster,
        n_geo_clusters=n_geo_clusters,
        use_geo_hour=use_geo_hour,
        use_geo_day=use_geo_day,
```

```

        use_district_hour=use_district_hour,
        use_float32=use_float32,
        C=C,
        max_iter=max_iter,
        alpha=alpha
    )

    fold_results.append({
        "fold": fold,
        "train_log_loss": result["train_log_loss"],
        "val_log_loss": result["test_log_loss"],
        "train_accuracy": result["train_accuracy"],
        "val_accuracy": result["test_accuracy"],
        "n_predictors": len(result["predictors"])
    })

    if verbose:
        print(
            f"Fold {fold} | "
            f"Val log loss: {result['test_log_loss']:.4f} | "
            f"Val accuracy: {result['test_accuracy']:.4f}"
        )

fold_results = pd.DataFrame(fold_results)

# Build summary dictionary
summary = {
    "model_type": model_type,
    "validation_type": validation_type,
    "selected_predictors": selected_predictors,
    "one_hot_encoding": one_hot_encoding,
    "drop_first": drop_first,
    "full_preprocessing": full_preprocessing,
    "use_interaction": use_interaction,
    "use_pca": use_pca,
    "pcs": pcs,
    "cyclical_encoding": cyclical_encoding,
    "use_geo_cluster": use_geo_cluster,
    "n_geo_clusters": n_geo_clusters if use_geo_cluster else None,
    "use_geo_hour": use_geo_hour,
    "use_geo_day": use_geo_day,
    "use_district_hour": use_district_hour,
    "use_float32": use_float32,
    "C": C if model_type == "logistic" else None,
    "alpha": alpha if model_type == "nb" else None,
    "test_size": test_size if validation_type == "holdout" else None,
    "n_splits": n_splits if validation_type == "kfold" else None,
    "n_predictors_mean": fold_results["n_predictors"].mean(),
    "val_log_loss_mean": fold_results["val_log_loss"].mean(),
    "val_log_loss_std": fold_results["val_log_loss"].std(ddof=0),
    "val_accuracy_mean": fold_results["val_accuracy"].mean(),
    "val_accuracy_std": fold_results["val_accuracy"].std(ddof=0),
    "train_log_loss_mean": fold_results["train_log_loss"].mean(),
    "train_accuracy_mean": fold_results["train_accuracy"].mean()
}

```

```

return summary, fold_results

```

```

In [120]: log_summary, log_folds = model_validation(
    X=X_train,
    y=y_train,
    model_type="logistic",
    selected_predictors=[],
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=False,
    cyclical_encoding=False,
    C=1.0,
    max_iter=500,
    validation_type="holdout",
    test_size=0.25
)

log_summary

Fold 1 | Val log loss: 2.5133 | Val accuracy: 0.2413

Out[120]: {'model_type': 'logistic',
'validation_type': 'holdout',
'selected_predictors': [],
'one_hot_encoding': True,
'drop_first': False,
'full_preprocessing': True,
'use_interaction': False,
'use_pca': False,
'pcs': [],
'cyclical_encoding': False,
'use_geo_cluster': False,
'n_geo_clusters': None,
'use_geo_hour': False,
'use_geo_day': False,
'use_district_hour': False,
'use_float32': True,
'C': 1.0,
'alpha': None,
'test_size': 0.25,
'n_splits': None,
'n_predictors_mean': np.float64(25.0),
'val_log_loss_mean': np.float64(2.513250297729414),
'val_log_loss_std': 0.0,
'val_accuracy_mean': np.float64(0.24134762866558337),
'val_accuracy_std': 0.0,
'train_log_loss_mean': np.float64(2.5103592614010433),
'train_accuracy_mean': np.float64(0.24059014252412736)}

```

```

In [121]: nb_summary, nb_folds = model_validation(
    X=X_train,
    y=y_train,
    model_type="nb",
    selected_predictors=[
        "hour",
        "day_of_week",

```

```

        "month",
        "pddistrict",
        "intersection",
        "block",
        "street_1",
        "street_2"
    ],
    one_hot_encoding=False,
    full_preprocessing=False,
    use_interaction=False,
    cyclical_encoding=False,
    alpha=1.0,
    validation_type="holdout",
    test_size=0.25
)

nb_summary

```

Fold 1 | Val log loss: 2.6309 | Val accuracy: 0.2392

```

Out[121]: {'model_type': 'nb',
'validation_type': 'holdout',
'selected_predictors': ['hour',
'day_of_week',
'month',
'pddistrict',
'intersection',
'block',
'street_1',
'street_2'],
'one_hot_encoding': False,
'drop_first': False,
'full_preprocessing': False,
'use_interaction': False,
'use_pca': False,
'pcs': [],
'cyclical_encoding': False,
'use_geo_cluster': False,
'n_geo_clusters': None,
'use_geo_hour': False,
'use_geo_day': False,
'use_district_hour': False,
'use_float32': True,
'C': None,
'alpha': 1.0,
'test_size': 0.25,
'n_splits': None,
'n_predictors_mean': np.float64(16.0),
'val_log_loss_mean': np.float64(2.6308572780812605),
'val_log_loss_std': 0.0,
'val_accuracy_mean': np.float64(0.23917599356102598),
'val_accuracy_std': 0.0,
'train_log_loss_mean': np.float64(2.6149052171909872),
'train_accuracy_mean': np.float64(0.24054205233730963)}

```

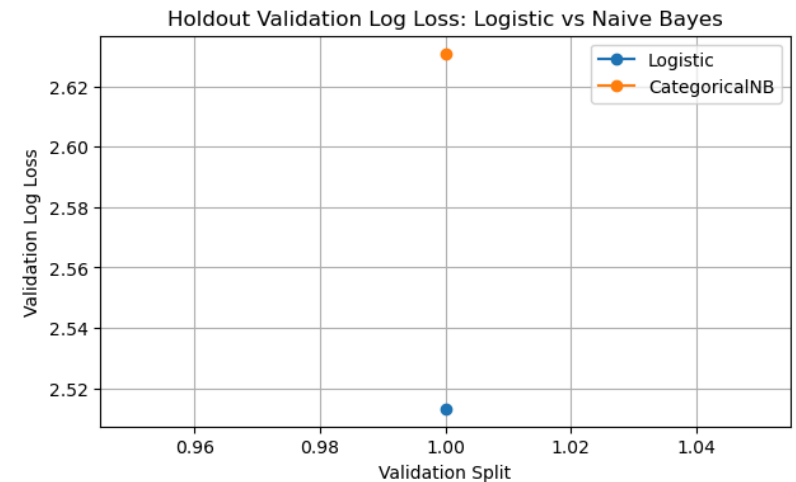
```
In [178]: plt.figure(figsize=(7, 4))
```

```

plt.plot(log_folds["fold"], log_folds["val_log_loss"], marker="o", label="Logistic")
plt.plot(nb_folds["fold"], nb_folds["val_log_loss"], marker="o", label="CategoricalNB")

plt.xlabel("Validation Split")
plt.ylabel("Validation Log Loss")
plt.title("Holdout Validation Log Loss: Logistic vs Naive Bayes")
plt.legend()
plt.grid()
plt.show()

```



8. Fast Holdout Experiments

```

In [131]: # A helper function for future testing
def compare_models(results_list, sort_by="val_log_loss_mean"):
    comparison_df = pd.DataFrame(results_list)

    comparison_df = comparison_df.sort_values(
        by=sort_by
    ).reset_index(drop=True)

    return comparison_df

```

```

In [176]: # Basic Log regression test
log_result = predictive_algo(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    model_type="logistic",
    one_hot_encoding=True,
    use_interaction=False,

```

```

    cyclical_encoding=False
)

log_result["test_log_loss"], log_result["test_accuracy"]

```

Out[176]: (2.512692479991928, 0.24041766208324197)

```

In [177]: #Basic NB test
nb_result = predictive_algo(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    model_type="nb",
    selected_predictors=[
        "hour",
        "day_of_week",
        "month",
        "pddistrict",
        "intersection",
        "block",
        "street_1",
        "street_2"
    ],
    one_hot_encoding=False,
    full_preprocessing=False,
    use_interaction=False,
    cyclical_encoding=False
)

nb_result["test_log_loss"], nb_result["test_accuracy"]

```

Out[177]: (2.6249538838409094, 0.23794039175040504)

```

In [124]: baseline_log, _ = model_validation(
    X=X_train,
    y=y_train,
    model_type="logistic",
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=False,
    cyclical_encoding=False,
    C=1.0,
    validation_type="holdout",
    test_size=0.25
)

```

Fold 1 | Val log loss: 2.5133 | Val accuracy: 0.2413

```

In [125]: cyclical_log, _ = model_validation(
    X=X_train,
    y=y_train,
    model_type="logistic",
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=False,

```

```

    cyclical_encoding=True,
    C=1.0,
    validation_type="holdout",
    test_size=0.25
)

```

Fold 1 | Val log loss: 2.4978 | Val accuracy: 0.2447

```

In [126]: interaction_log, _ = model_validation(
    X=X_train,
    y=y_train,
    model_type="logistic",
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=True,
    cyclical_encoding=False,
    C=1.0,
    validation_type="holdout",
    test_size=0.25
)

```

Fold 1 | Val log loss: 2.5100 | Val accuracy: 0.2421

```

In [127]: combined_log, _ = model_validation(
    X=X_train,
    y=y_train,
    model_type="logistic",
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=True,
    cyclical_encoding=True,
    C=1.0,
    validation_type="holdout",
    test_size=0.25
)

```

Fold 1 | Val log loss: 2.4947 | Val accuracy: 0.2445

```

In [129]: c_results = []

for c in [0.01, 0.1, 1, 3, 10]:

    result, _ = model_validation(
        X=X_train,
        y=y_train,
        model_type="logistic",
        one_hot_encoding=True,
        full_preprocessing=True,
        use_interaction=True,
        cyclical_encoding=True,
        C=c,
        validation_type="holdout",
        test_size=0.25
    )

    result["experiment"] = f"C={c}"

```

```
c_results.append(result)
```

```
Fold 1 | Val log loss: 2.4948 | Val accuracy: 0.2446
Fold 1 | Val log loss: 2.4946 | Val accuracy: 0.2446
Fold 1 | Val log loss: 2.4947 | Val accuracy: 0.2445
Fold 1 | Val log loss: 2.4947 | Val accuracy: 0.2445
Fold 1 | Val log loss: 2.4948 | Val accuracy: 0.2446
```

```
In [133]: comparison_df = compare_models([
    baseline_log,
    cyclical_log,
    interaction_log,
    combined_log
])

comparison_df[
    [
        "val_log_loss_mean",
        "val_accuracy_mean",
        "use_interaction",
        "cyclical_encoding"
    ]
]
```

```
Out[133]:
```

	val_log_loss_mean	val_accuracy_mean	use_interaction	cyclical_encoding
0	2.494716	0.244529	True	True
1	2.497850	0.244711	False	True
2	2.510044	0.242107	True	False
3	2.513250	0.241348	False	False

```
In [179]: plt.figure(figsize=(8, 4))

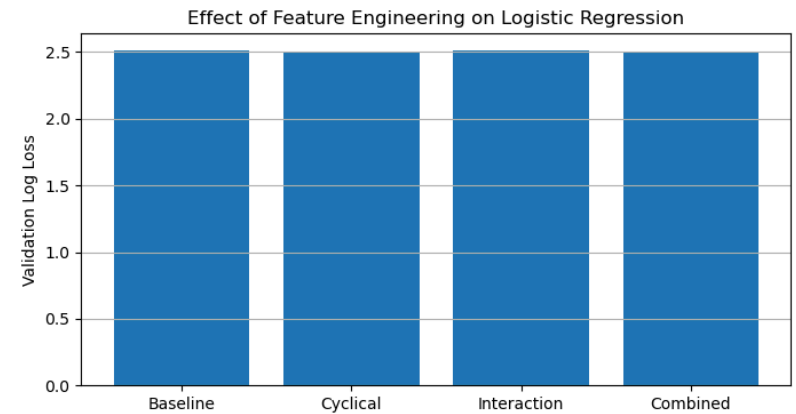
labels = [
    "Baseline",
    "Cyclical",
    "Interaction",
    "Combined"
]

losses = [
    baseline_log["val_log_loss_mean"],
    cyclical_log["val_log_loss_mean"],
    interaction_log["val_log_loss_mean"],
    combined_log["val_log_loss_mean"]
]

plt.bar(labels, losses)

plt.ylabel("Validation Log Loss")
plt.title("Effect of Feature Engineering on Logistic Regression")
```

```
plt.grid(axis="y")
plt.show()
```

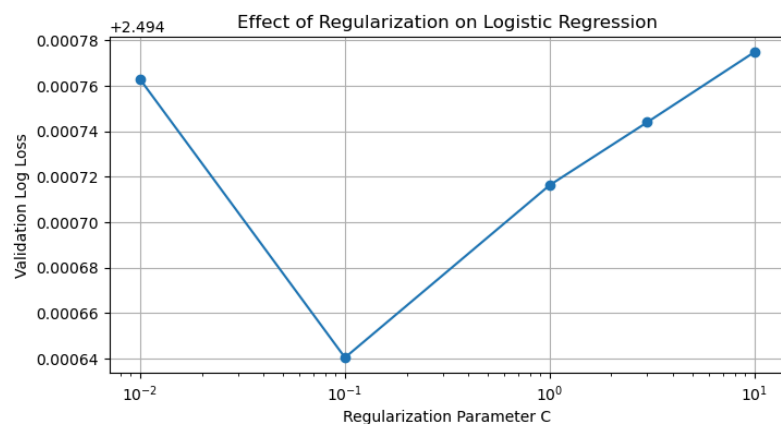


```
In [138]: c_df = pd.DataFrame(c_results)

plt.figure(figsize=(8, 4))

plt.plot(
    c_df["C"],
    c_df["val_log_loss_mean"],
    marker="o"
)

plt.xscale("log")
plt.xlabel("Regularization Parameter C")
plt.ylabel("Validation Log Loss")
plt.title("Effect of Regularization on Logistic Regression")
plt.grid()
plt.show()
```



```
In [136]: geo_log, _ = model_validation(
    X=X_train,
    y=y_train,
    model_type="logistic",
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=True,
    cyclical_encoding=True,
    use_geo_cluster=True,
    n_geo_clusters=20,
    C=1.0,
    max_iter=500,
    validation_type="holdout",
    test_size=0.25
)

geo_log
```

C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8

warnings.warn(

Fold 1 | Val log loss: 2.4660 | Val accuracy: 0.2538

```
Out[136]: {'model_type': 'logistic',
'validation_type': 'holdout',
'selected_predictors': [],
'one_hot_encoding': True,
'drop_first': False,
'full_preprocessing': True,
'use_interaction': True,
'use_pca': False,
'pcs': [],
'cyclical_encoding': True,
'use_geo_cluster': True,
'n_geo_clusters': 20,
'use_geo_hour': False,
'use_geo_day': False,
'use_district_hour': False,
'use_float32': True,
'C': 1.0,
'alpha': None,
'test_size': 0.25,
'n_splits': None,
'n_predictors_mean': np.float64(61.0),
'val_log_loss_mean': np.float64(2.465975699792776),
'val_log_loss_std': 0.0,
'val_accuracy_mean': np.float64(0.2537623957842944),
'val_accuracy_std': 0.0,
'train_log_loss_mean': np.float64(2.4592496699779964),
'train_accuracy_mean': np.float64(0.254108547144835)}
```

```
In [140]: geo_results = []

for k in [10, 20, 30, 40]:

    result, _ = model_validation(
        X=X_train,
        y=y_train,
        model_type="logistic",
        one_hot_encoding=True,
        full_preprocessing=True,
        use_interaction=True,
        cyclical_encoding=True,
        use_geo_cluster=True,
        n_geo_clusters=k,
        C=1.0,
        max_iter=500,
        validation_type="holdout",
        test_size=0.25
    )

    result["experiment"] = f"geo_clusters={k}"
    geo_results.append(result)

geo_df[
    [
        "experiment",
        "n_geo_clusters",
        "val_log_loss_mean",
```

```

        "val_log_loss_std",
        "val_accuracy_mean"
    ]
].sort_values("val_log_loss_mean")

```

```

C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

```

```

Fold 1 | Val log loss: 2.4842 | Val accuracy: 0.2470

```

```

C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

```

```

Fold 1 | Val log loss: 2.4660 | Val accuracy: 0.2538

```

```

C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

```

```

Fold 1 | Val log loss: 2.4605 | Val accuracy: 0.2553

```

```

C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

```

```

Fold 1 | Val log loss: 2.4510 | Val accuracy: 0.2581

```

```

Out[140]:

```

	experiment	n_geo_clusters	val_log_loss_mean	val_log_loss_std	val_accuracy_mean
3	geo_clusters=40	40	2.450981	0.0	0.258128
2	geo_clusters=30	30	2.460477	0.0	0.255281
1	geo_clusters=20	20	2.465976	0.0	0.253762
0	geo_clusters=10	10	2.484229	0.0	0.246982

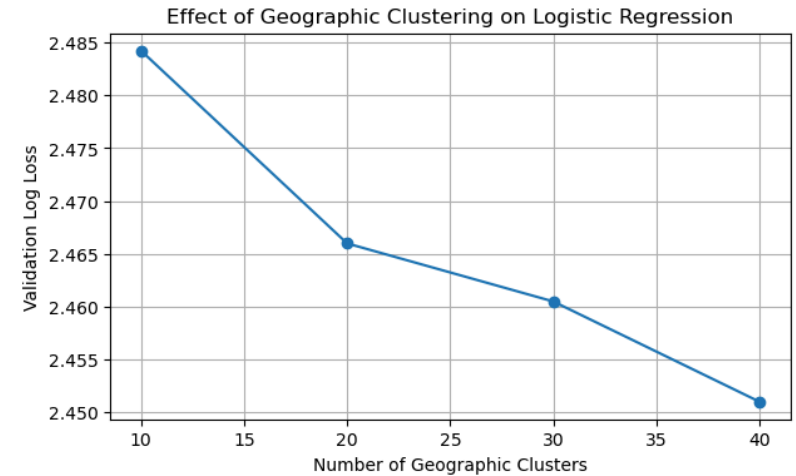
```

In [141]: plt.figure(figsize=(7, 4))

plt.plot(
    geo_df["n_geo_clusters"],
    geo_df["val_log_loss_mean"],
    marker="o"
)

plt.xlabel("Number of Geographic Clusters")
plt.ylabel("Validation Log Loss")
plt.title("Effect of Geographic Clustering on Logistic Regression")
plt.grid()
plt.show()

```



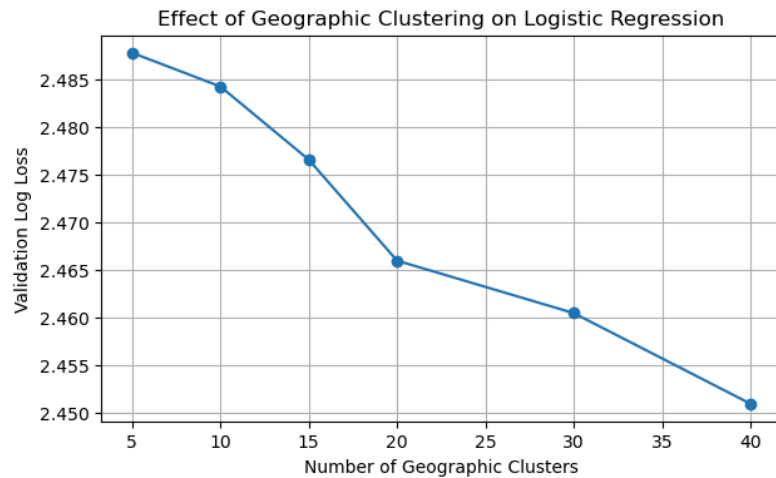
```

In [143]: plt.figure(figsize=(7, 4))

plt.plot(
    geo_df["n_geo_clusters"],
    geo_df["val_log_loss"],
    marker="o"
)

plt.xlabel("Number of Geographic Clusters")
plt.ylabel("Validation Log Loss")
plt.title("Effect of Geographic Clustering on Logistic Regression")
plt.grid()
plt.show()

```



In [144]: interaction_feature_results = []

```
configs = [
    {
        "name": "geo_only",
        "use_geo_hour": False,
        "use_geo_day": False,
        "use_district_hour": False
    },
    {
        "name": "geo_hour",
        "use_geo_hour": True,
        "use_geo_day": False,
        "use_district_hour": False
    },
    {
        "name": "geo_day",
        "use_geo_hour": False,
        "use_geo_day": True,
        "use_district_hour": False
    },
    {
        "name": "district_hour",
        "use_geo_hour": False,
        "use_geo_day": False,
        "use_district_hour": True
    },
    {
        "name": "geo_hour_geo_day",
        "use_geo_hour": True,
        "use_geo_day": True,
        "use_district_hour": False
    },
]
```

```
{
    "name": "all_targeted_interactions",
    "use_geo_hour": True,
    "use_geo_day": True,
    "use_district_hour": True
}

]

for cfg in configs:

    result, _ = model_validation(
        X=X_train,
        y=y_train,
        model_type="logistic",
        validation_type="holdout",
        one_hot_encoding=True,
        full_preprocessing=True,
        use_interaction=True,
        cyclical_encoding=True,
        use_geo_cluster=True,
        n_geo_clusters=20,
        use_geo_hour=cfg["use_geo_hour"],
        use_geo_day=cfg["use_geo_day"],
        use_district_hour=cfg["use_district_hour"],
        C=1.0,
        max_iter=500,
        test_size=0.25
    )

    result["experiment"] = cfg["name"]
    interaction_feature_results.append(result)

targeted_df = pd.DataFrame(interaction_feature_results)

targeted_df[
    [
        "experiment",
        "n_predictors_mean",
        "val_log_loss_mean",
        "val_log_loss_std",
        "val_accuracy_mean"
    ]
].sort_values("val_log_loss_mean")
```

C:\Users\ao\lve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8

warnings.warn(

Fold 1 | Val log loss: 2.4660 | Val accuracy: 0.2538


```
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:201: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_test[interaction_name] = X_test[col] * X_test['intersection']
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:200: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_train[interaction_name] = X_train[col] * X_train['intersection']
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:201: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_test[interaction_name] = X_test[col] * X_test['intersection']
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:200: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_train[interaction_name] = X_train[col] * X_train['intersection']
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:201: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_test[interaction_name] = X_test[col] * X_test['intersection']
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:200: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_train[interaction_name] = X_train[col] * X_train['intersection']
C:\Users\aoive\AppData\Local\Temp\ipykernel_21492\209624554.py:201: PerformanceWarnin
g: DataFrame is highly fragmented. This is usually the result of calling `frame.inser
t` many times, which has poor performance. Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`
X_test[interaction_name] = X_test[col] * X_test['intersection']
Fold 1 | Val log loss: 2.4943 | Val accuracy: 0.2561
```

Out[144]:

	experiment	n_predictors_mean	val_log_loss_mean	val_log_loss_std	val_accu
3	district_hour	301.0	2.465620	0.0	
0	geo_only	61.0	2.465976	0.0	
2	geo_day	201.0	2.470145	0.0	
1	geo_hour	541.0	2.481821	0.0	
4	geo_hour_geo_day	681.0	2.484020	0.0	
5	all_targeted_interactions	921.0	2.494337	0.0	

In [145]: *# Fast A/B test: with vs without cyclical encoding.*
Keep everything else fixed so the comparison is fair.

```
cyclical_test_results = []
```

```
for use_cyc in [False, True]:

    result, _ = model_validation(
        X=X_train,
        y=y_train,
        model_type="logistic",
        validation_type="holdout",
        one_hot_encoding=True,
        full_preprocessing=True,
        use_interaction=True,
        cyclical_encoding=use_cyc,
        use_geo_cluster=True,
        n_geo_clusters=40,
        C=1.0,
        max_iter=500,
        test_size=0.25
    )

    result["experiment"] = "cyclical_on" if use_cyc else "cyclical_off"
    cyclical_test_results.append(result)
```

```
cyclical_test_df = pd.DataFrame(cyclical_test_results)
```

```
cyclical_test_df[
    [
        "experiment",
        "n_predictors_mean",
        "val_log_loss_mean",
        "val_log_loss_std",
        "val_accuracy_mean",
        "train_log_loss_mean"
    ]
].sort_values("val_log_loss_mean")
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
```

```
Fold 1 | Val log loss: 2.4654 | Val accuracy: 0.2566
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
```

```
Fold 1 | Val log loss: 2.4510 | Val accuracy: 0.2581
```

Out[145]:

	experiment	n_predictors_mean	val_log_loss_mean	val_log_loss_std	val_accuracy_mean
1	cyclical_on	81.0	2.450981	0.0	0.258128
0	cyclical_off	75.0	2.465400	0.0	0.256572

```
In [146]: c_results = []
```

```
for c in [0.005, 0.01, 0.03, 0.05, 0.1, 0.2, 0.3]:
```

```
    result, _ = model_validation(  
        X=X_train,  
        y=y_train,  
        model_type="logistic",  
        validation_type="holdout",  
        one_hot_encoding=True,  
        full_preprocessing=True,  
        use_interaction=True,  
        cyclical_encoding=True,  
        use_geo_cluster=True,  
        n_geo_clusters=40,  
        C=c,  
        max_iter=500,  
        test_size=0.25  
    )
```

```
    result["experiment"] = f"C={c}"  
    c_results.append(result)
```

```
c_df = pd.DataFrame(c_results)
```

```
c_df[  
    [  
        "experiment",  
        "val_log_loss_mean",  
        "val_accuracy_mean",  
        "train_log_loss_mean"  
    ]  
].sort_values("val_log_loss_mean")
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4522 | Val accuracy: 0.2576
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4516 | Val accuracy: 0.2575
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4513 | Val accuracy: 0.2575
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4510 | Val accuracy: 0.2578
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4508 | Val accuracy: 0.2580
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4509 | Val accuracy: 0.2581
```

```
C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p  
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK  
L, when there are less chunks than available threads. You can prevent it by setting ba  
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8  
warnings.warn(  
Fold 1 | Val log loss: 2.4509 | Val accuracy: 0.2580
```

```
Out[146]:
```

	experiment	val_log_loss_mean	val_accuracy_mean	train_log_loss_mean
4	C=0.1	2.450834	0.258007	2.443260
5	C=0.2	2.450913	0.258068	2.443058
6	C=0.3	2.450947	0.258022	2.443005
3	C=0.05	2.451042	0.257817	2.443681
2	C=0.03	2.451341	0.257513	2.444458
1	C=0.01	2.451578	0.257529	2.445241
0	C=0.005	2.452245	0.257574	2.446417

```
In [147]: nb_alpha_results = []
```

```
for a in [0.01, 0.05, 0.1, 0.5, 1, 2, 5, 10]:
```

```
    result, _ = model_validation(  
        X=X_train,  
        y=y_train,  
        model_type="nb",  
        validation_type="holdout",  
        selected_predictors=[  
            "hour",  
            "day_of_week",  
            "month",  
            "pddistrict",  
            "intersection",  
        ],  
    )
```

```

        "block",
        "street_1",
        "street_2"
    ],
    one_hot_encoding=False,
    full_preprocessing=False,
    use_interaction=False,
    cyclical_encoding=False,
    alpha=a,
    test_size=0.25
)

result["experiment"] = f"alpha={a}"
result["alpha"] = a
nb_alpha_results.append(result)

nb_alpha_df = pd.DataFrame(nb_alpha_results)

nb_alpha_df[
    [
        "experiment",
        "val_log_loss_mean",
        "val_accuracy_mean",
        "train_log_loss_mean"
    ]
].sort_values("val_log_loss_mean")

```

```

Fold 1 | Val log loss: 2.6385 | Val accuracy: 0.2346
Fold 1 | Val log loss: 2.6359 | Val accuracy: 0.2351
Fold 1 | Val log loss: 2.6342 | Val accuracy: 0.2354
Fold 1 | Val log loss: 2.6300 | Val accuracy: 0.2375
Fold 1 | Val log loss: 2.6309 | Val accuracy: 0.2392
Fold 1 | Val log loss: 2.6364 | Val accuracy: 0.2411
Fold 1 | Val log loss: 2.6557 | Val accuracy: 0.2446
Fold 1 | Val log loss: 2.6819 | Val accuracy: 0.2478

```

```
Out[147]:
```

	experiment	val_log_loss_mean	val_accuracy_mean	train_log_loss_mean
3	alpha=0.5	2.630014	0.237475	2.613752
4	alpha=1	2.630857	0.239176	2.614905
2	alpha=0.1	2.634159	0.235372	2.617340
1	alpha=0.05	2.635937	0.235091	2.618919
5	alpha=2	2.636421	0.241135	2.620855
0	alpha=0.01	2.638481	0.234643	2.621031
6	alpha=5	2.655742	0.244605	2.640895
7	alpha=10	2.681861	0.247764	2.667791

```
In [148]: plt.figure(figsize=(7, 4))
```

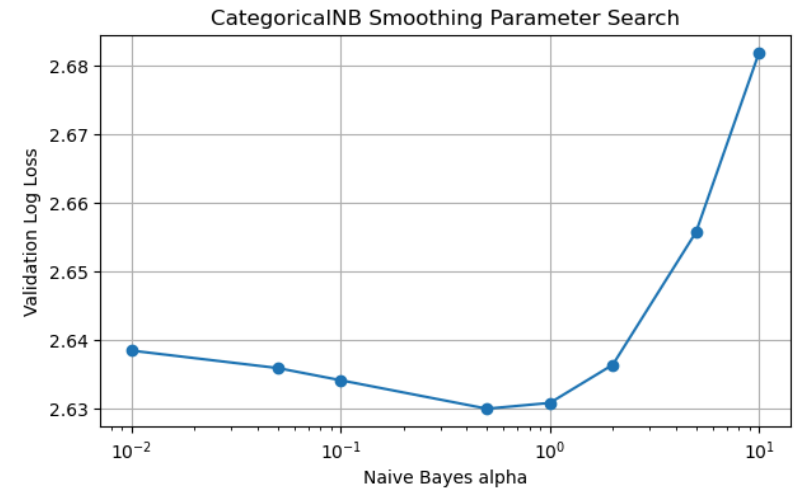
```
plt.plot(
    nb_alpha_df["alpha"],
```

```

nb_alpha_df["val_log_loss_mean"],
marker="o"
)

plt.xscale("log")
plt.xlabel("Naive Bayes alpha")
plt.ylabel("Validation Log Loss")
plt.title("CategoricalNB Smoothing Parameter Search")
plt.grid()
plt.show()

```



```
In [149]: # Naive Bayes model selection search
```

```

nb_feature_sets = {
    "time_only": [
        "hour", "day_of_week", "month"
    ],

    "time_district": [
        "hour", "day_of_week", "month",
        "pddistrict"
    ],

    "time_district_address": [
        "hour", "day_of_week", "month",
        "pddistrict",
        "intersection", "block", "street_1", "street_2"
    ],

    "time_district_location": [
        "hour", "day_of_week", "month",
        "pddistrict",
        "x", "y"
    ]
}

```

```

],
    "full_nb_candidate": [
        "hour", "day_of_week", "month",
        "pddistrict",
        "intersection", "block", "street_1", "street_2",
        "x", "y"
    ]
}

nb_results = []

for feature_name, features in nb_feature_sets.items():

    for a in [0.01, 0.05, 0.1, 0.5, 1, 2, 5, 10]:

        result, _ = model_validation(
            X=X_train,
            y=y_train,
            model_type="nb",
            validation_type="holdout",
            selected_predictors=features,
            one_hot_encoding=False,
            full_preprocessing=False,
            use_interaction=False,
            cyclical_encoding=False,
            use_geo_cluster=False,
            alpha=a,
            test_size=0.25
        )

        result["experiment"] = feature_name
        result["alpha_value"] = a
        nb_results.append(result)

nb_df = pd.DataFrame(nb_results)

nb_df[
    [
        "experiment",
        "alpha_value",
        "n_predictors_mean",
        "val_log_loss_mean",
        "val_accuracy_mean",
        "train_log_loss_mean"
    ]
].sort_values("val_log_loss_mean").head(15)

```

Fold 1		Val log loss: 2.6385		Val accuracy: 0.2346
Fold 1		Val log loss: 2.6359		Val accuracy: 0.2351
Fold 1		Val log loss: 2.6342		Val accuracy: 0.2354
Fold 1		Val log loss: 2.6300		Val accuracy: 0.2375
Fold 1		Val log loss: 2.6309		Val accuracy: 0.2392
Fold 1		Val log loss: 2.6364		Val accuracy: 0.2411
Fold 1		Val log loss: 2.6557		Val accuracy: 0.2446
Fold 1		Val log loss: 2.6819		Val accuracy: 0.2478
Fold 1		Val log loss: 2.6385		Val accuracy: 0.2346
Fold 1		Val log loss: 2.6359		Val accuracy: 0.2351
Fold 1		Val log loss: 2.6342		Val accuracy: 0.2354
Fold 1		Val log loss: 2.6300		Val accuracy: 0.2375
Fold 1		Val log loss: 2.6309		Val accuracy: 0.2392
Fold 1		Val log loss: 2.6364		Val accuracy: 0.2411
Fold 1		Val log loss: 2.6557		Val accuracy: 0.2446
Fold 1		Val log loss: 2.6819		Val accuracy: 0.2478
Fold 1		Val log loss: 2.6385		Val accuracy: 0.2346
Fold 1		Val log loss: 2.6359		Val accuracy: 0.2351
Fold 1		Val log loss: 2.6342		Val accuracy: 0.2354
Fold 1		Val log loss: 2.6300		Val accuracy: 0.2375
Fold 1		Val log loss: 2.6309		Val accuracy: 0.2392
Fold 1		Val log loss: 2.6364		Val accuracy: 0.2411
Fold 1		Val log loss: 2.6557		Val accuracy: 0.2446
Fold 1		Val log loss: 2.6819		Val accuracy: 0.2478
Fold 1		Val log loss: 2.6385		Val accuracy: 0.2346
Fold 1		Val log loss: 2.6359		Val accuracy: 0.2351
Fold 1		Val log loss: 2.6342		Val accuracy: 0.2354
Fold 1		Val log loss: 2.6300		Val accuracy: 0.2375
Fold 1		Val log loss: 2.6309		Val accuracy: 0.2392
Fold 1		Val log loss: 2.6364		Val accuracy: 0.2411
Fold 1		Val log loss: 2.6557		Val accuracy: 0.2446
Fold 1		Val log loss: 2.6819		Val accuracy: 0.2478
Fold 1		Val log loss: 2.6385		Val accuracy: 0.2346
Fold 1		Val log loss: 2.6359		Val accuracy: 0.2351
Fold 1		Val log loss: 2.6342		Val accuracy: 0.2354
Fold 1		Val log loss: 2.6300		Val accuracy: 0.2375
Fold 1		Val log loss: 2.6309		Val accuracy: 0.2392
Fold 1		Val log loss: 2.6364		Val accuracy: 0.2411
Fold 1		Val log loss: 2.6557		Val accuracy: 0.2446
Fold 1		Val log loss: 2.6819		Val accuracy: 0.2478

Out[149]:

	experiment	alpha_value	n_predictors_mean	val_log_loss_mean	val_accuracy_mean
3	time_only	0.5	16.0	2.630014	0.2374
11	time_district	0.5	16.0	2.630014	0.2374
27	time_district_location	0.5	16.0	2.630014	0.2374
19	time_district_address	0.5	16.0	2.630014	0.2374
35	full_nb_candidate	0.5	16.0	2.630014	0.2374
20	time_district_address	1.0	16.0	2.630857	0.2391
12	time_district	1.0	16.0	2.630857	0.2391
28	time_district_location	1.0	16.0	2.630857	0.2391
36	full_nb_candidate	1.0	16.0	2.630857	0.2391
4	time_only	1.0	16.0	2.630857	0.2391
2	time_only	0.1	16.0	2.634159	0.2351
10	time_district	0.1	16.0	2.634159	0.2351
26	time_district_location	0.1	16.0	2.634159	0.2351
34	full_nb_candidate	0.1	16.0	2.634159	0.2351
18	time_district_address	0.1	16.0	2.634159	0.2351

In [150]:

```
nb_geo_results = []

for k in [10, 20, 40]:

    for a in [0.1, 0.5, 1, 2, 5]:

        result, _ = model_validation(
            X=X_train,
            y=y_train,
            model_type="nb",
            validation_type="holdout",
            selected_predictors=[
                "hour", "day_of_week", "month",
                "pddistrict",
                "intersection", "block", "street_1", "street_2"
            ],
            one_hot_encoding=False,
            full_preprocessing=False,
            use_interaction=False,
            cyclical_encoding=False,
            use_geo_cluster=True,
            n_geo_clusters=k,
            alpha=a,
            test_size=0.25
        )
```

```
result["experiment"] = f"nb_geo_{k}"
result["alpha_value"] = a
nb_geo_results.append(result)

nb_geo_df = pd.DataFrame(nb_geo_results)

nb_geo_df[
    [
        "experiment",
        "n_geo_clusters",
        "alpha_value",
        "n_predictors_mean",
        "val_log_loss_mean",
        "val_accuracy_mean",
        "train_log_loss_mean"
    ]
].sort_values("val_log_loss_mean").head(15)
```

C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

Fold 1 | Val log loss: 2.7587 | Val accuracy: 0.2281

C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

Fold 1 | Val log loss: 2.7488 | Val accuracy: 0.2306

C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

Fold 1 | Val log loss: 2.7458 | Val accuracy: 0.2323

C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

Fold 1 | Val log loss: 2.7468 | Val accuracy: 0.2347

C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

Fold 1 | Val log loss: 2.7590 | Val accuracy: 0.2390

C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(

Fold 1 | Val log loss: 2.8010 | Val accuracy: 0.2273

```
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7902 | Val accuracy: 0.2299
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7867 | Val accuracy: 0.2320
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7870 | Val accuracy: 0.2348
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7980 | Val accuracy: 0.2386
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7233 | Val accuracy: 0.2330
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7165 | Val accuracy: 0.2349
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7155 | Val accuracy: 0.2367
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
Fold 1 | Val log loss: 2.7185 | Val accuracy: 0.2386
C:\Users\ao\ve\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster\_kmeans.p
y:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MK
L, when there are less chunks than available threads. You can prevent it by setting ba
tch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8
warnings.warn(
```

Fold 1 | Val log loss: 2.7329 | Val accuracy: 0.2426

Out[150]:

	experiment	n_geo_clusters	alpha_value	n_predictors_mean	val_log_loss_mean	val_acc
12	nb_geo_40	40	1.0	17.0	2.715467	
11	nb_geo_40	40	0.5	17.0	2.716513	
13	nb_geo_40	40	2.0	17.0	2.718453	
10	nb_geo_40	40	0.1	17.0	2.723319	
14	nb_geo_40	40	5.0	17.0	2.732931	
2	nb_geo_10	10	1.0	17.0	2.745751	
3	nb_geo_10	10	2.0	17.0	2.746755	
1	nb_geo_10	10	0.5	17.0	2.748754	
0	nb_geo_10	10	0.1	17.0	2.758699	
4	nb_geo_10	10	5.0	17.0	2.758957	
7	nb_geo_20	20	1.0	17.0	2.786682	
8	nb_geo_20	20	2.0	17.0	2.787046	
6	nb_geo_20	20	0.5	17.0	2.790175	
9	nb_geo_20	20	5.0	17.0	2.798036	
5	nb_geo_20	20	0.1	17.0	2.800988	

```
In [151]: best_nb_plot_df = nb_df.copy()

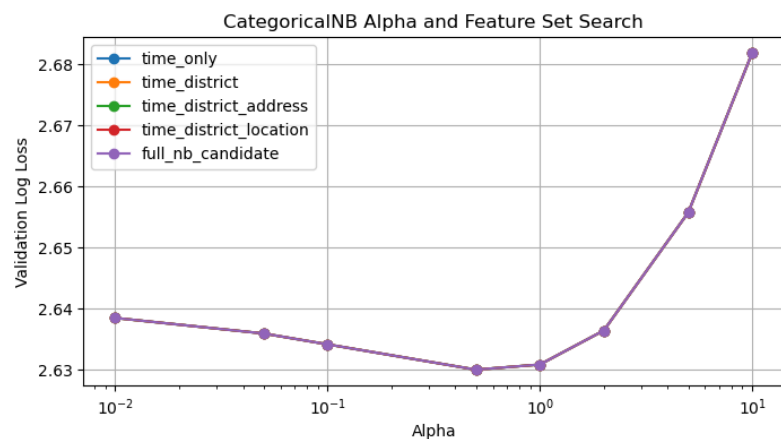
plt.figure(figsize=(8, 4))

for name in best_nb_plot_df["experiment"].unique():

    temp = best_nb_plot_df[best_nb_plot_df["experiment"] == name]

    plt.plot(
        temp["alpha_value"],
        temp["val_log_loss_mean"],
        marker="o",
        label=name
    )

plt.xscale("log")
plt.xlabel("Alpha")
plt.ylabel("Validation Log Loss")
plt.title("CategoricalNB Alpha and Feature Set Search")
plt.legend()
plt.grid()
plt.show()
```

```
In [152]: final_log_result = predictive_algo(
    X_train=X_train,
    X_test=X_test,
    y_train=y_train,
    y_test=y_test,
    model_type="logistic",
    one_hot_encoding=True,
    full_preprocessing=True,
    use_interaction=True,
    cyclical_encoding=True,
    use_geo_cluster=True,
    n_geo_clusters=40,
    use_float32=True,
    C=0.1,
    max_iter=500
)
```

final_log_result

C:\Users\aoive\miniconda3\envs\pydata-book\lib\site-packages\sklearn\cluster_kmeans.py:1955: UserWarning: MiniBatchKMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can prevent it by setting batch_size >= 2560 or by setting the environment variable OMP_NUM_THREADS=8

warnings.warn(

```
Out[152]: {'model_type': 'logistic',
  'model': LogisticRegression(C=0.1, max_iter=500),
  'predictors': ['x',
    'y',
    'year',
    'month',
    'day',
    'hour',
    'minute',
    'dt_numeric',
    'day_of_week',
    'day_of_year',
    'week_of_year',
    'intersection',
    'street_1',
    'street_2',
    'block',
    'hour_sin',
    'hour_cos',
    'day_of_week_sin',
    'day_of_week_cos',
    'month_sin',
    'month_cos',
    'pddistrict_BAYVIEW',
    'pddistrict_CENTRAL',
    'pddistrict_INGLESIDE',
    'pddistrict_MISSION',
    'pddistrict_NORTHERN',
    'pddistrict_PARK',
    'pddistrict_RICHMOND',
    'pddistrict_SOUTHERN',
    'pddistrict_TARAVAL',
    'pddistrict_TENDERLOIN',
    'geo_cluster_0',
    'geo_cluster_1',
    'geo_cluster_2',
    'geo_cluster_3',
    'geo_cluster_4',
    'geo_cluster_5',
    'geo_cluster_6',
    'geo_cluster_7',
    'geo_cluster_8',
    'geo_cluster_9',
    'geo_cluster_10',
    'geo_cluster_11',
    'geo_cluster_12',
    'geo_cluster_13',
    'geo_cluster_14',
    'geo_cluster_15',
    'geo_cluster_16',
    'geo_cluster_17',
    'geo_cluster_18',
    'geo_cluster_19',
    'geo_cluster_20',
    'geo_cluster_21',
    'geo_cluster_22',
```

```

'geo_cluster_23',
'geo_cluster_24',
'geo_cluster_25',
'geo_cluster_26',
'geo_cluster_27',
'geo_cluster_28',
'geo_cluster_29',
'geo_cluster_30',
'geo_cluster_31',
'geo_cluster_32',
'geo_cluster_33',
'geo_cluster_34',
'geo_cluster_35',
'geo_cluster_36',
'geo_cluster_37',
'geo_cluster_38',
'geo_cluster_39',
'pddistrict_BAYVIEW:intersection',
'pddistrict_CENTRAL:intersection',
'pddistrict_INGLESIDE:intersection',
'pddistrict_MISSION:intersection',
'pddistrict_NORTHERN:intersection',
'pddistrict_PARK:intersection',
'pddistrict_RICHMOND:intersection',
'pddistrict_SOUTHERN:intersection',
'pddistrict_TARAVAL:intersection',
'pddistrict_TENDERLOIN:intersection'],
'label_encoder': LabelEncoder(),
'train_log_loss': 2.4585458052592064,
'test_log_loss': 2.4633154813691216,
'train_accuracy': 0.25140616072089583,
'test_accuracy': 0.24985691628249992,
'y_train_proba': array([[9.61919432e-03, 1.45902428e-01, 2.53363863e-03, ...,
1.13338733e-01, 3.69937769e-02, 1.22008402e-02],
[5.31311827e-03, 8.94793034e-02, 2.93350870e-04, ...,
1.05749309e-01, 4.74070903e-02, 1.29005236e-02],
[1.42223727e-02, 1.70234934e-01, 9.51555544e-04, ...,
7.22889338e-02, 5.92738536e-02, 1.96558389e-02],
...,
[3.92618770e-03, 7.78020758e-02, 9.06284582e-07, ...,
8.51898523e-02, 3.56069423e-02, 1.89933416e-02],
[1.98491469e-03, 8.17438310e-02, 1.19828094e-05, ...,
4.75510674e-02, 3.20438654e-02, 8.43075849e-03],
[1.01157691e-03, 3.97215307e-02, 7.54085741e-07, ...,
3.42776413e-02, 3.82208573e-02, 9.20037777e-03]],
shape=(526789, 39)),
'y_test_proba': array([[8.43299828e-03, 1.09734170e-01, 5.64446883e-03, ...,
1.68686275e-01, 1.19059536e-02, 3.62965424e-03],
[1.21507658e-03, 5.54534670e-02, 4.66506149e-04, ...,
1.35889183e-01, 1.88552669e-02, 2.87376257e-03],
[2.52992354e-03, 5.87238937e-02, 3.41008973e-04, ...,
1.00892197e-01, 3.89002176e-02, 4.71034108e-03],
...,
[4.91460795e-03, 6.34196229e-02, 8.64299883e-06, ...,
3.76879206e-02, 1.75865624e-02, 7.85771769e-03],
[1.59600257e-03, 5.12015795e-02, 5.22326310e-07, ...,

```

```

2.87923661e-02, 7.72024173e-02, 1.45296374e-02],
[1.59600257e-03, 5.12015795e-02, 5.22326310e-07, ...,
2.87923661e-02, 7.72024173e-02, 1.45296374e-02]],
shape=(351193, 39)),
'y_train_pred': array([ 1, 21, 1, ..., 21, 16, 16], shape=(526789,), dtype=int8),
'y_test_pred': array([36, 16, 16, ..., 16, 16, 16], shape=(351193,), dtype=int8)}

```

```

In [153]: final_nb_result = predictive_algo(
X_train=X_train,
X_test=X_test,
y_train=y_train,
y_test=y_test,
model_type="nb",
one_hot_encoding=False,
full_preprocessing=False,
selected_predictors=[
    "hour",
    "day_of_week",
    "month",
    "pddistrict",
    "intersection",
    "block",
    "street_1",
    "street_2"
],
cyclical_encoding=False,
alpha=0.5
)

final_nb_result

```

```
Out[153]: {'model_type': 'nb',
'model': CategoricalNB(alpha=0.5),
'predictors': ['pddistrict',
'x',
'y',
'year',
'month',
'day',
'hour',
'minute',
'dt_numeric',
'day_of_week',
'day_of_year',
'week_of_year',
'intersection',
'street_1',
'street_2',
'block'],
'label_encoder': LabelEncoder(),
'train_log_loss': 2.61569812266643,
'test_log_loss': 2.6250046007228374,
'train_accuracy': 0.23854712228235594,
'test_accuracy': 0.23681850150771797,
'y_train_proba': array([[2.39149831e-02, 9.46946583e-02, 2.32620364e-03, ...,
1.39550663e-01, 1.70160700e-02, 1.11264424e-02],
[5.62974148e-03, 5.95641015e-02, 5.95089665e-05, ...,
1.99603889e-01, 3.75413534e-02, 8.24479681e-03],
[4.67413617e-02, 1.43251211e-01, 5.89792689e-04, ...,
1.18564289e-01, 2.97770203e-02, 2.15125706e-02],
...,
[1.69596309e-03, 2.96226877e-02, 1.02527505e-08, ...,
2.87677705e-01, 1.28295254e-02, 1.24776863e-02],
[2.46832545e-03, 3.37577263e-02, 1.47197218e-06, ...,
6.59391951e-02, 2.28132032e-02, 2.72016360e-03],
[4.90411126e-04, 2.11351958e-02, 2.97539203e-08, ...,
2.36812555e-02, 3.56843868e-02, 5.73951389e-03]],
shape=(526789, 39)),
'y_test_proba': array([[9.15446526e-03, 8.55774102e-02, 4.15738898e-03, ...,
2.28622542e-01, 1.25412302e-02, 4.12823487e-03],
[1.21299631e-03, 4.61019398e-02, 3.07419103e-04, ...,
4.75600485e-02, 8.83206776e-02, 4.72243542e-03],
[2.57917093e-04, 1.98466298e-02, 5.33683739e-05, ...,
5.03912812e-02, 8.11951725e-02, 1.97710926e-03],
...,
[4.27165237e-03, 5.40442593e-02, 2.28478846e-06, ...,
4.01641207e-02, 9.99485984e-03, 2.99083707e-03],
[8.08670631e-04, 1.96961584e-02, 3.41361891e-08, ...,
4.52535726e-02, 1.00612914e-01, 6.20103636e-03],
[8.08670631e-04, 1.96961584e-02, 3.41361891e-08, ...,
4.52535726e-02, 1.00612914e-01, 6.20103636e-03]],
shape=(351193, 39)),
'y_train_pred': array([19, 21, 19, ..., 21, 16, 16], shape=(526789,), dtype=int8),
'y_test_pred': array([36, 21, 21, ..., 16, 21, 21], shape=(351193,), dtype=int8)}
```

11. Final Model Evaluation

After selecting the strongest configurations from holdout experiments, the final models are trained on the training set and evaluated once on the held-out test set. This separates model selection from final test performance.

```
In [154]: final_comparison = pd.DataFrame([
{
"model": "Optimized Logistic",
"test_log_loss": final_log_result["test_log_loss"],
"test_accuracy": final_log_result["test_accuracy"],
"train_log_loss": final_log_result["train_log_loss"],
"n_predictors": len(final_log_result["predictors"])
},
{
"model": "Optimized Naive Bayes",
"test_log_loss": final_nb_result["test_log_loss"],
"test_accuracy": final_nb_result["test_accuracy"],
"train_log_loss": final_nb_result["train_log_loss"],
"n_predictors": len(final_nb_result["predictors"])
}
])

final_comparison.sort_values("test_log_loss")
```

```
Out[154]:
```

	model	test_log_loss	test_accuracy	train_log_loss	n_predictors
0	Optimized Logistic	2.463315	0.249857	2.458546	81
1	Optimized Naive Bayes	2.625005	0.236819	2.615698	16

12. Visualization Analysis

```
In [159]: # Use the full cleaned dataset.
df_viz = sf_crime(full=True, encoding=False, datetime_index=False).copy()

df_viz["date_time"] = pd.to_datetime(df_viz["date_time"])

# date columns.
df_viz["date"] = df_viz["date_time"].dt.date
df_viz["year"] = df_viz["date_time"].dt.year
df_viz["month"] = df_viz["date_time"].dt.month
df_viz["hour"] = df_viz["date_time"].dt.hour
df_viz["day_of_week"] = df_viz["date_time"].dt.dayofweek
df_viz["week"] = df_viz["date_time"].dt.isocalendar().week.astype(int)

# Use the 10 most frequent categories
top_categories = df_viz["category"].value_counts().head(10).index.tolist()

df_top = df_viz[df_viz["category"].isin(top_categories)].copy()

top_categories
```

```
Out[159]: ['LARCENY/THEFT',
           'OTHER OFFENSES',
           'NON-CRIMINAL',
           'ASSAULT',
           'DRUG/NARCOTIC',
           'VEHICLE THEFT',
           'VANDALISM',
           'WARRANTS',
           'BURGLARY',
           'SUSPICIOUS OCC']
```

12.1 Temporal Crime Patterns by Category

```
In [167]: hourly_counts = (
    df_top
    .groupby(["date", "hour", "category"])
    .size()
    .reset_index(name="count")
)

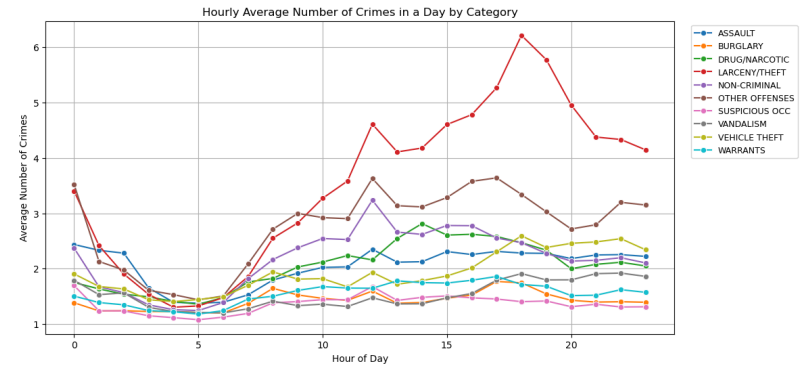
hourly_avg = hourly_counts.sort_values(["category", "hour"])

hourly_avg = (
    hourly_counts
    .groupby(["hour", "category"])["count"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=hourly_avg,
    x="hour",
    y="count",
    hue="category",
    marker="o"
)

plt.title("Hourly Average Number of Crimes in a Day by Category")
plt.xlabel("Hour of Day")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.legend(
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    fontsize=9
)
plt.show()
```



```
In [166]: daily_counts = (
    df_top
    .groupby(["date", "day_of_week", "category"])
    .size()
    .reset_index(name="count")
)

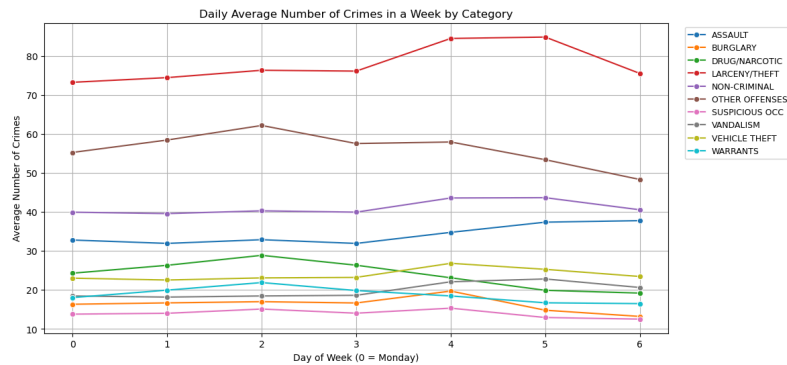
daily_avg = (
    daily_counts
    .groupby(["day_of_week", "category"])["count"]
    .mean()
    .reset_index()
)

daily_avg = daily_avg.sort_values(["category", "day_of_week"])

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=daily_avg,
    x="day_of_week",
    y="count",
    hue="category",
    marker="o"
)

plt.title("Daily Average Number of Crimes in a Week by Category")
plt.xlabel("Day of Week (0 = Monday)")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.legend(
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    fontsize=9
)
plt.show()
```



In [165]: `weekly_df = df_top[(df_top["year"] >= 2004) & (df_top["year"] <= 2014)].copy()`

```
weekly_df = weekly_df[
    (weekly_df["week"] >= 1) &
    (weekly_df["week"] <= 52)
]

weekly_counts = (
    weekly_df
    .groupby(["year", "week", "category"])
    .size()
    .reset_index(name="count")
)

weekly_avg = weekly_avg.sort_values(["category", "week"])

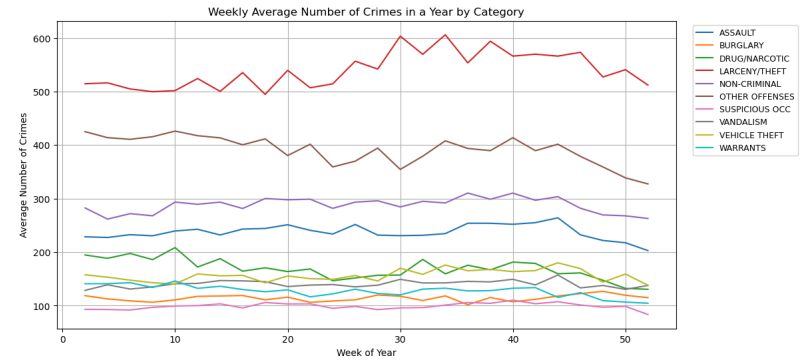
weekly_avg = (
    weekly_counts
    .groupby(["week", "category"])["count"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=weekly_avg,
    x="week",
    y="count",
    hue="category"
)

plt.title("Weekly Average Number of Crimes in a Year by Category")
plt.xlabel("Week of Year")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.legend(
    bbox_to_anchor=(1.02, 1),
```

```
loc="upper left",
fontsize=9
)
plt.show()
```



In [164]: `# Keep full years only so every month is fully represented.`
`monthly_df = df_top[(df_top["year"] >= 2004) & (df_top["year"] <= 2014)].copy()`

```
monthly_df = df_top[
    (df_top["year"] >= 2004) &
    (df_top["year"] <= 2014)
].copy()

monthly_counts = (
    monthly_df
    .groupby(["year", "month", "category"])
    .size()
    .reset_index(name="count")
)

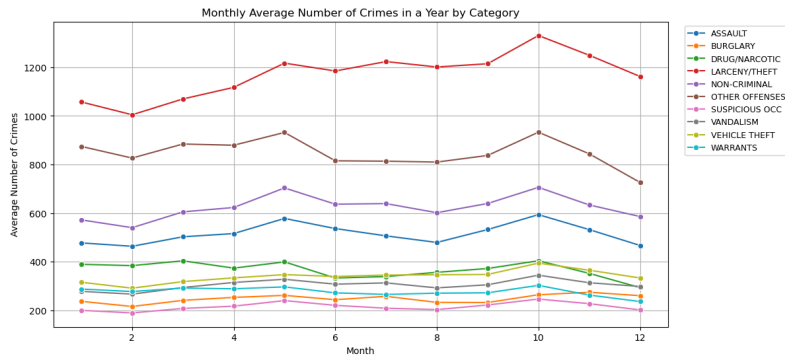
monthly_avg = (
    monthly_counts
    .groupby(["month", "category"])["count"]
    .mean()
    .reset_index()
)

monthly_avg = monthly_avg.sort_values(["category", "month"])

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=monthly_avg,
    x="month",
    y="count",
    hue="category",
    marker="o"
)
```

```
plt.title("Monthly Average Number of Crimes in a Year by Category")
plt.xlabel("Month")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.legend(
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    fontsize=9
)
plt.show()
```



In [168]:

```
yearly_df = df_top[(df_top["year"] >= 2004) & (df_top["year"] <= 2014)].copy()
```

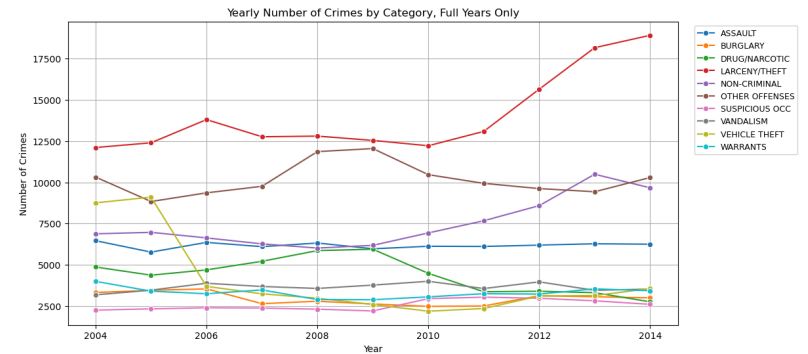
```
yearly_counts = (
    yearly_df
    .groupby(["year", "category"])
    .size()
    .reset_index(name="count")
)
```

```
plt.figure(figsize=(12, 6))
```

```
sns.lineplot(
    data=yearly_counts,
    x="year",
    y="count",
    hue="category",
    marker="o"
)
```

```
plt.title("Yearly Number of Crimes by Category, Full Years Only")
plt.xlabel("Year")
plt.ylabel("Number of Crimes")
plt.grid()
plt.legend(
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    fontsize=9
)
```

```
)
plt.show()
```



12.2 Overall Temporal Crime Patterns

In [169]:

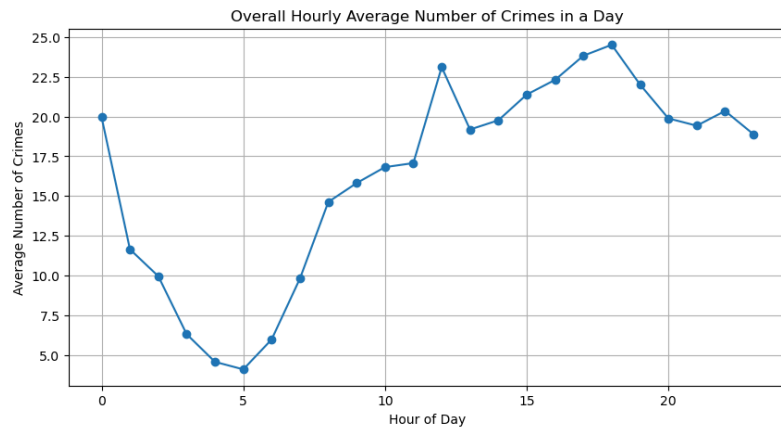
```
hourly_all_counts = (
    df_viz
    .groupby(["date", "hour"])
    .size()
    .reset_index(name="count")
)
```

```
hourly_all_avg = (
    hourly_all_counts
    .groupby("hour")["count"]
    .mean()
    .reset_index()
)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(
    hourly_all_avg["hour"],
    hourly_all_avg["count"],
    marker="o"
)
```

```
plt.title("Overall Hourly Average Number of Crimes in a Day")
plt.xlabel("Hour of Day")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.show()
```



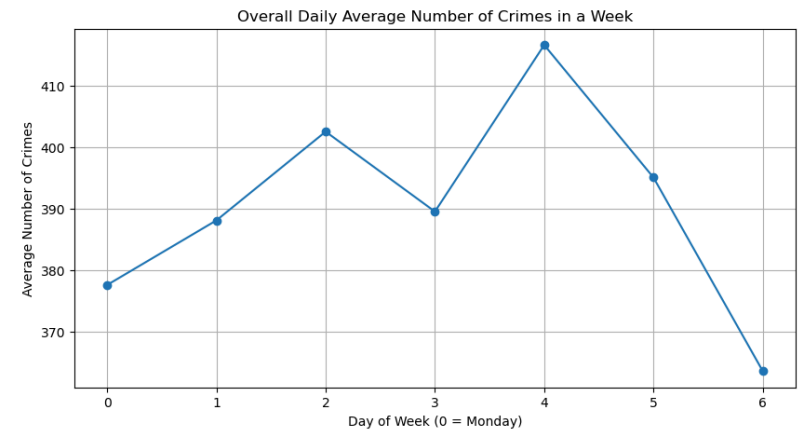
```
In [170]: daily_all_counts = (
    df_viz
    .groupby(["date", "day_of_week"])
    .size()
    .reset_index(name="count")
)

daily_all_avg = (
    daily_all_counts
    .groupby("day_of_week")["count"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(10, 5))

plt.plot(
    daily_all_avg["day_of_week"],
    daily_all_avg["count"],
    marker="o"
)

plt.title("Overall Daily Average Number of Crimes in a Week")
plt.xlabel("Day of Week (0 = Monday)")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.show()
```



```
In [171]: weekly_all_df = df_viz[(df_viz["year"] >= 2004) & (df_viz["year"] <= 2014)].copy()

weekly_all_df = weekly_all_df[
    (weekly_all_df["week"] >= 1) &
    (weekly_all_df["week"] <= 52)
]

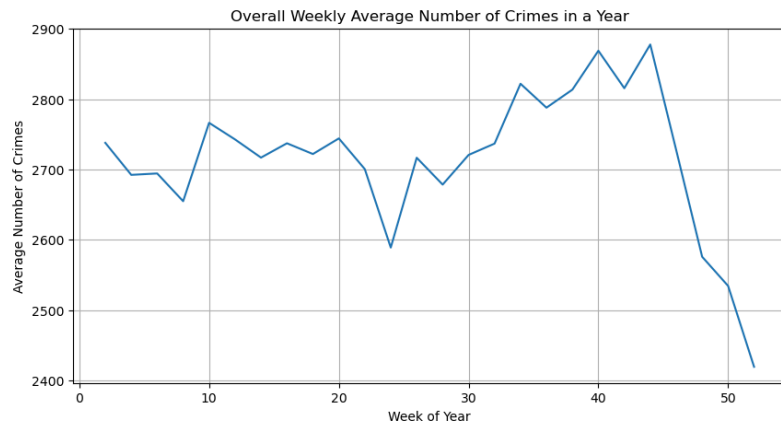
weekly_all_counts = (
    weekly_all_df
    .groupby(["year", "week"])
    .size()
    .reset_index(name="count")
)

weekly_all_avg = (
    weekly_all_counts
    .groupby("week")["count"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(10, 5))

plt.plot(
    weekly_all_avg["week"],
    weekly_all_avg["count"]
)

plt.title("Overall Weekly Average Number of Crimes in a Year")
plt.xlabel("Week of Year")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.show()
```



```
In [172]: monthly_all_df = df_viz[(df_viz["year"] >= 2004) & (df_viz["year"] <= 2014)].copy()

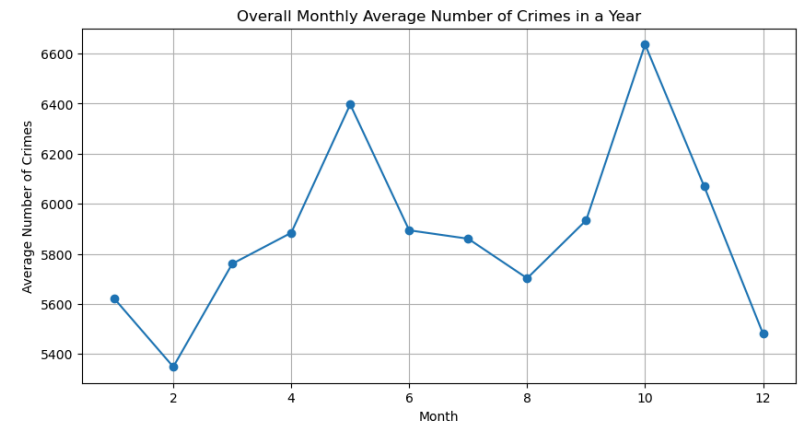
monthly_all_counts = (
    monthly_all_df
    .groupby(["year", "month"])
    .size()
    .reset_index(name="count")
)

monthly_all_avg = (
    monthly_all_counts
    .groupby("month")["count"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(10, 5))

plt.plot(
    monthly_all_avg["month"],
    monthly_all_avg["count"],
    marker="o"
)

plt.title("Overall Monthly Average Number of Crimes in a Year")
plt.xlabel("Month")
plt.ylabel("Average Number of Crimes")
plt.grid()
plt.show()
```



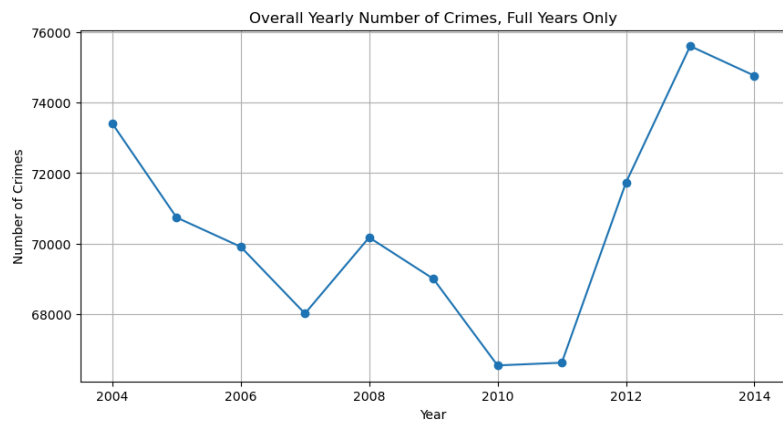
```
In [173]: yearly_all_df = df_viz[(df_viz["year"] >= 2004) & (df_viz["year"] <= 2014)].copy()

yearly_all_counts = (
    yearly_all_df
    .groupby("year")
    .size()
    .reset_index(name="count")
)

plt.figure(figsize=(10, 5))

plt.plot(
    yearly_all_counts["year"],
    yearly_all_counts["count"],
    marker="o"
)

plt.title("Overall Yearly Number of Crimes, Full Years Only")
plt.xlabel("Year")
plt.ylabel("Number of Crimes")
plt.grid()
plt.show()
```

12.3 Geographic Density and Neighborhood Structure

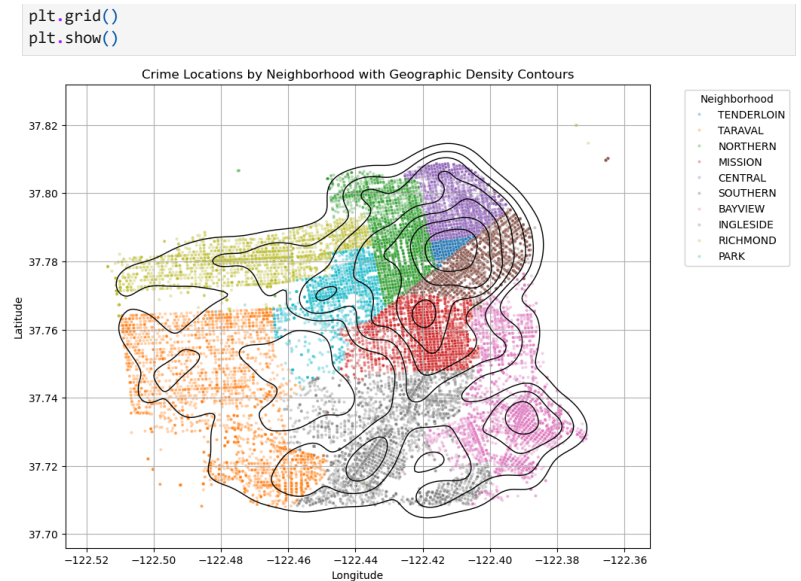
```
In [174]: # Simple random sample of 50,000 observations
scatter_sample = df_viz.sample(
    n=50000,
    random_state=12345
)

plt.figure(figsize=(10, 8))

# Density contour curves
sns.kdeplot(
    data=scatter_sample,
    x="x",
    y="y",
    levels=8,
    color="black",
    linewidths=1
)

# Crime Locations colored by neighborhood / police district
sns.scatterplot(
    data=scatter_sample,
    x="x",
    y="y",
    hue="pddistrict",
    s=8,
    alpha=0.35
)

plt.title("Crime Locations by Neighborhood with Geographic Density Contours")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend(title="Neighborhood", bbox_to_anchor=(1.05, 1), loc="upper left")
```



12.4 Neighborhood Crime Composition

```
In [175]: # Simple random sample of 200,000 observations
bar_sample = df_viz.sample(
    n=200000,
    random_state=12345
)

# Identify the 5 most frequent crimes in the sample
top_5_crimes = bar_sample["category"].value_counts().head(5).index.tolist()

# Keep only top 5 crimes
bar_top5 = bar_sample[bar_sample["category"].isin(top_5_crimes)].copy()

# Count crimes by neighborhood and category
crime_counts = (
    bar_top5
    .groupby(["pddistrict", "category"])
    .size()
    .reset_index(name="count")
)

# Convert counts to proportions within each neighborhood
crime_counts["proportion"] = (
    crime_counts["count"] /
    crime_counts.groupby("pddistrict")["count"].transform("sum")
)
```

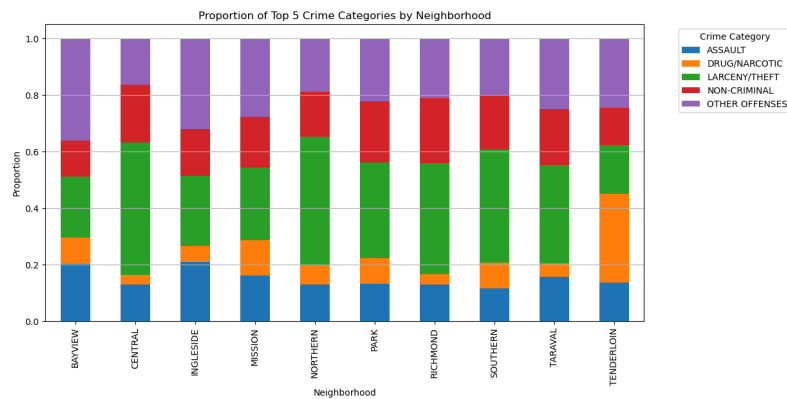
```

stacked_df = crime_counts.pivot(
    index="pddistrict",
    columns="category",
    values="proportion"
).fillna(0)

# Plot
stacked_df.plot(
    kind="bar",
    stacked=True,
    figsize=(12, 6)
)

plt.title("Proportion of Top 5 Crime Categories by Neighborhood")
plt.xlabel("Neighborhood")
plt.ylabel("Proportion")
plt.legend(title="Crime Category", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.grid(axis="y")
plt.show()

```



In []:

13. Key Findings and Next Steps

The strongest model was regularized multinomial logistic regression with cyclical temporal encoding and geographic clustering. Geographic clustering produced the largest improvement, suggesting that crime category prediction depends strongly on localized spatial structure. Categorical Naive Bayes performed worse, likely because the conditional independence assumption is unrealistic for correlated temporal, spatial, and district features.

Future work will explore higher-order logistic regression interactions, learned spatial-temporal embeddings, neural-network feature extraction, and hybrid models that use final hidden-layer activations as engineered inputs for downstream logistic regression.